Bulk RNA-seq: technical details and transcript usage

Lieven Clement Koen Van den Berge

```
library(tidyverse)
library(SummarizedExperiment)
library(edgeR)
library(BiocFileCache)
library(tximeta)
library(org.Hs.eg.db)
library(DESeq2)
library(stageR)
library(patchwork)
library(macrophage)
```

In the introduction to bulk RNA-seq we introduced count models and a complete edgeR workflow. In this chapter we look under the hood:

1. How generalised linear models (GLMs) are fitted and how inference works.
2. How edgeR estimates the variability with negative binomial (NB) and quasi-likelihood (QL) models, and how this compares to limma-voom and DESeq2.
3. The calibration of the tests under the null hypothesis.
4. Normalisation and independent filtering.
5. Analyses at the transcript level: differential transcript expression and differential transcript usage.

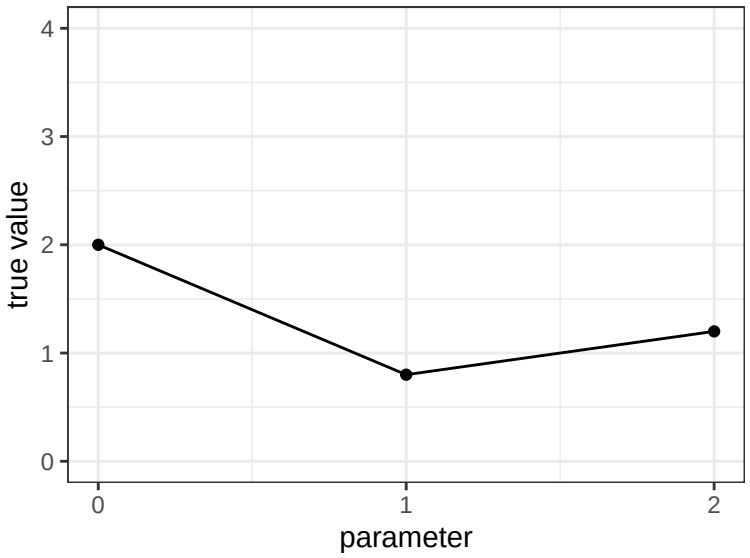
Generalised linear models

Simulated Poisson data

We simulate Poisson counts for 100 observations with two covariates, $\beta_0 = 2$, $\beta_1 = 0.8$ and $\beta_2 = 1.2$.

```r
set.seed(300)
X <- cbind(1, rnorm(100), rnorm(100))
betasTrue <- c(2, 0.8, 1.2)
etaTrue <- X %*% betasTrue
y <- rpois(100, exp(etaTrue))
```

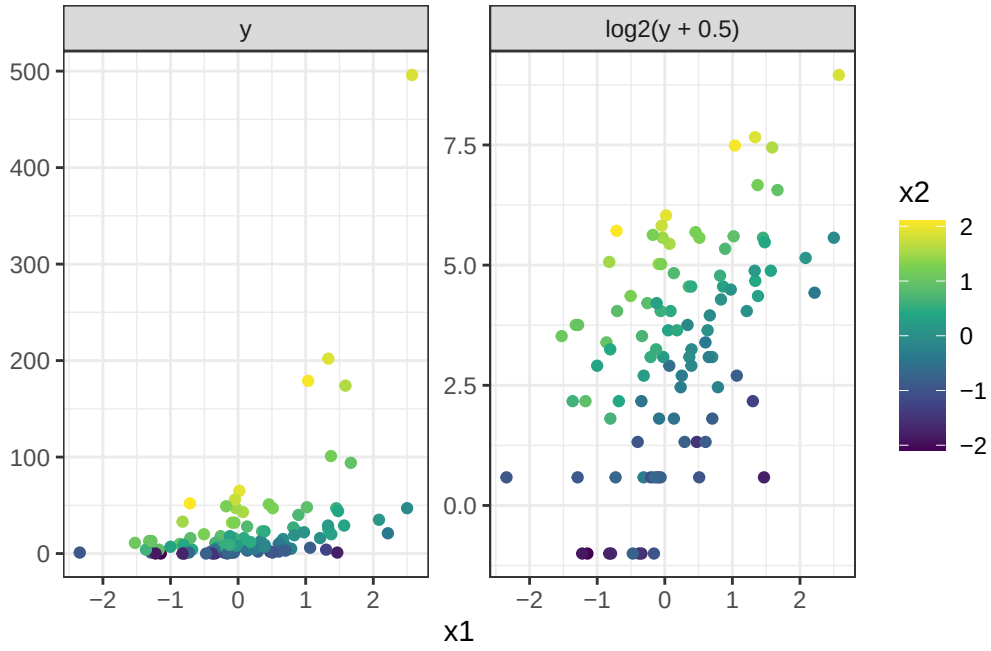
```r
tibble(coef = 0:2, beta = betasTrue) |>
  ggplot(aes(coef, beta)) +
  geom_point() +
  geom_line() +
  scale_x_continuous(breaks = 0:2) +
  ylim(0, 4) +
  xlab("parameter") +
  ylab("true value") +
  theme_bw()
```



The counts increase exponentially with the covariates; on the log scale the relation becomes (approximately) linear.

```r
simData <- tibble(x1 = X[, 2], x2 = X[, 3], y = y)
if (knitr::is_html_output()) {
  plot3d <- function(z, title) {
    plotly::plot_ly(simData, x = ~x1, y = ~x2, z = z, size = 0.5) |>
      plotly::add_markers(type = "scatter3d") |>
      plotly::layout(title = title, scene = list(aspectmode = "cube", zaxis = list(title = ti
```

```
  }
  htmltools::tagList(
    plot3d(~y, "y"),
    plot3d(~ log2(y + 0.5), "log2(y + 0.5)")
  )
} else {
  simData |>
    mutate(`log2(y + 0.5)` = log2(y + 0.5)) |>
    pivot_longer(c(y, `log2(y + 0.5)`), names_to = "scale", values_to = "count") |>
    mutate(scale = factor(scale, levels = c("y", "log2(y + 0.5)"))) |>
    ggplot(aes(x1, count, color = x2)) +
    geom_point() +
    facet_wrap(~scale, scales = "free_y") +
    scale_color_viridis_c() +
    ylab("") +
    theme_bw()
}
```



Exponential family

GLMs assume that the response follows a distribution of the exponential family:

$$f(y_i|\theta_i, \phi) = \exp\left\{\frac{y_i\theta_i - b(\theta_i)}{a(\phi)} + c(y_i, \phi)\right\}$$

with

- θ_i: the canonical parameter,
- ϕ: the dispersion parameter,
- $a(.)$, $b(.)$, $c(.)$: functions that depend on the distribution.

The first two moments follow from $b(.)$:

- $E[y_i] = \mu_i = b'(\theta_i)$,
- $\text{Var}[y_i] = a(\phi)\, b''(\theta_i)$.

The variance is thus a function of the mean.

Poisson.

$$f(y_i|\mu_i) = \frac{\mu_i^{y_i} e^{-\mu_i}}{y_i!} = \exp\left\{y_i \log(\mu_i) - \mu_i - \log(y_i!)\right\}$$

so

- $\theta_i = \log \mu_i$,
- $b(\theta_i) = \exp(\theta_i)$,
- $a(\phi) = 1$ and
- $c(y_i, \phi) = -\log(y_i!)$.

Hence

- $E[y_i] = b'(\theta_i) = \exp(\theta_i) = \mu_i$ and
- $\text{Var}[y_i] = b''(\theta_i) = \mu_i$.

Components of a GLM

$$\begin{cases} y_i|\mathbf{x}_i & \sim & f(y_i|\theta_i, \phi) \\ E\left[y_i|\mathbf{x}_i\right] & = & \mu_i \\ g(\mu_i) & = & \eta_i \\ \eta_i & = & \mathbf{x}_i^T\beta \end{cases}$$

with $g(.)$ the link function, e.g.

- identity link $g(\mu) = \mu$ for the normal distribution,
- log link $g(\mu) = \log \mu$ for the Poisson distribution,
- logit link $g(\mu) = \log \frac{\mu}{1-\mu}$ for the Bernoulli distribution.

These are the **canonical** links: with a canonical link $\theta_i = \eta_i$.

Likelihood

- The likelihood is the probability (density) to observe the sample, as a function of the model parameters (here we explicitly assume that the observations are independent):

$$L(\beta|\mathbf{y}) = \prod_{i=1}^{n} f(y_i|\theta_i, \phi)$$

- We work with the log-likelihood

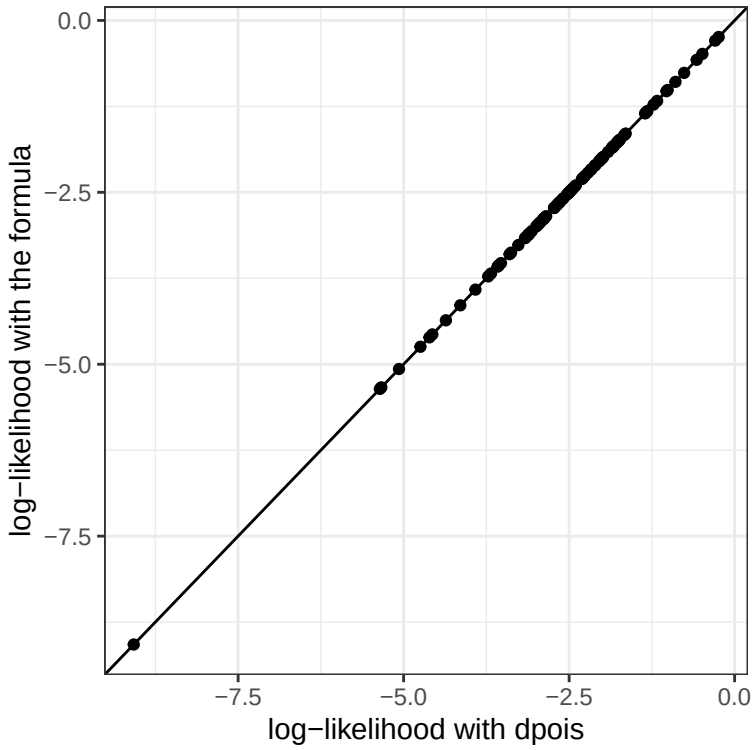
$$l(\beta|\mathbf{y}) = \sum_{i=1}^{n} \log f(y_i|\theta_i, \phi).$$

- For the Poisson GLM with log link:

$$l(\beta|\mathbf{y}) = \sum_{i=1}^{n} \left\{ y_i \eta_i - e^{\eta_i} - \log y_i! \right\}, \quad \theta_i = \eta_i = \mathbf{x}_i^T \beta.$$

We verify the formula on the simulated data, first for every observation and then for the full sample:

```
tibble(
  dpois = dpois(y, exp(etaTrue), log = TRUE),
  formula = c(y * etaTrue - exp(etaTrue) - lfactorial(y))
) |>
  ggplot(aes(dpois, formula)) +
  geom_point() +
  geom_abline(intercept = 0, slope = 1) +
  xlab("log-likelihood with dpois") +
  ylab("log-likelihood with the formula") +
  theme_bw()
```




```
c(dpois = sum(dpois(y, exp(etaTrue), log = TRUE)),
  formula = sum(y * etaTrue - exp(etaTrue) - lfactorial(y)))
```

```
   dpois  formula
-260.485 -260.485
```

Maximum likelihood estimation

We choose the parameters that maximise the log-likelihood, i.e. we solve the **score equations**

$$S(\beta) = \frac{\partial l(\beta|\mathbf{y})}{\partial \beta} = \mathbf{0}.$$

With the chain rule we obtain

$$
\begin{aligned}
S(\beta) &= \sum_{i=1}^{n} \frac{\partial l(\beta|y_i)}{\partial \beta} \\
&\Updownarrow \text{ canonical link} \\
&= \sum_{i=1}^{n} \frac{\partial \left\{ \frac{y_i\theta_i - b(\theta_i)}{a(\phi)} + c(y_i,\phi) \right\}}{\partial \theta_i} \frac{\partial \theta_i}{\partial \mu_i} \frac{\partial \mu_i}{\partial \eta_i} \frac{\partial \eta_i}{\partial \beta} \\
&= \sum_{i=1}^{n} \frac{y_i - \mu_i}{a(\phi)} \frac{1}{b''(\theta)} \frac{\partial \mu_i}{\partial \eta_i} \mathbf{x}_i^T \\
&= \sum_{i=1}^{n} \frac{y_i - \mu_i}{\text{Var}[y_i]} \frac{\partial \mu_i}{\partial \eta_i} \mathbf{x}_i^T \\
&= \mathbf{X}^T \mathbf{A} \left(\mathbf{y} - \mu \right),
\end{aligned}
$$

with $\mathbf{A}$ a diagonal matrix with elements $a_{ii} = \left(\text{Var}[y_i] \frac{\partial \eta_i}{\partial \mu_i} \right)^{-1}$.

- For the Poisson GLM with log link:
  - $\text{Var}[y_i] = \mu_i$ and
  - $\partial \mu_i / \partial \eta_i = \mu_i$, so $\mathbf{A} = \mathbf{I}$ and
  - $S(\beta) = \mathbf{X}^T(\mathbf{y} - \mu)$.

Note that,

- only the **mean and the variance** of y_i enter the score equations. This is the basis of quasi-likelihood.
- The score equations are non-linear in β because $\mu = g(\eta)$.
- $a_{ii} = \left(\text{var}[y_i] \frac{\partial \eta_i}{\partial \mu_i} \right)^{-1}$

$\rightarrow$ Find roots of score equation by using Newton-Raphson method.

Newton-Raphson and Fisher scoring

We find the root of the score function iteratively with the Newton-Raphson algorithm: walk along the tangent of the score function to the point where it crosses zero.

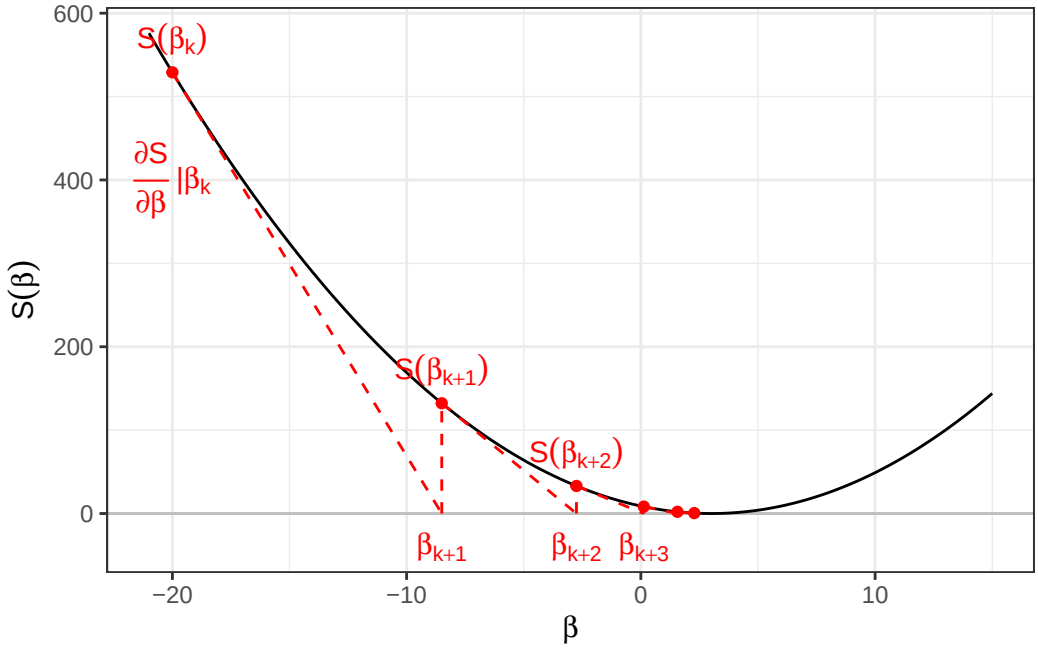


Figure 1: Newton-Raphson: starting from an initial guess, the algorithm walks along the tangent of the score function to the point where the tangent crosses zero, and repeats this until the root is found.

Newton Raphson algorithm to find the root of the score function.

1. Choose initial parameter estimate $\beta^k = \beta^0$
2. Calculate score $S(\beta)|_{\beta=\beta^k}$
3. Calculate derivative of the function for which you want to calculate the roots
4. Walk along first derivative until line (plane) of the derivative crosses zero
5. Update the betas β^{k+1}
6. Iterate from step 2 - 5 until convergence.

Derivative of Score

We have to implement an interative algorithm for optimisation. To make things tractable we will act as if $\mathbf{A}$ is known and fix it using the current values of β^k. Note, that for Poisson regression $\mathbf{A} = \mathbf{I}$.

$$
\begin{aligned}
\frac{\partial S(\beta)}{\partial \beta} &= \frac{\partial \mathbf{X}^T \mathbf{A} \{\mathbf{Y} - \mu\}}{\partial \beta} \\
&\approx -\mathbf{X}^T \mathbf{A} \begin{bmatrix} \frac{\partial \mu_1}{\partial \eta_1} & 0 & ... & 0 \\ 0 & \frac{\partial \mu_2}{\partial \eta_2} & ... & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & ... & \frac{\partial \mu_n}{\partial \eta_n} \end{bmatrix} \frac{\partial \eta}{\partial \beta} \\
&\approx -\mathbf{X}^T \mathbf{W} \mathbf{X}
\end{aligned}
$$

Define equation of Tangent Line (Plane)

- We know two points of the tangent plane $(\beta^k, S(\beta^k))$ and $(\beta^{k+1}, 0)$
- We know the direction of the plane $S'(\beta) = \frac{\partial S(\beta)}{\partial \beta}$
- Equation of Plane:

$$S(\beta) = \alpha_0 + S'|_{\beta^k} \beta$$

- Get β_{k+1}

$$
\begin{aligned}
\mathbf{0} &= \alpha_0 + S'|_{\beta^k} \beta^{k+1} \\
\beta^{k+1} &= -\left(S'|_{\beta^k}\right)^{-1} \alpha_0
\end{aligned}
$$

- Get α_0

$$
\begin{aligned}
S(\beta^k) &= \alpha_0 + S'|_{\beta^k} \beta^k \\
\alpha_0 &= -S'|_{\beta^k} \beta^k + S(\beta^k)
\end{aligned}
$$

- Get β_{k+1}

$$
\begin{aligned}
\beta^{k+1} &= \beta^k - \left(S'|_{\beta^k}\right)^{-1} S(\beta^k) \\
\beta^{k+1} &= \beta^k + \left(\mathbf{X}^T \mathbf{W} \mathbf{X}\right)^{-1} S(\beta^k)
\end{aligned}
$$

- $\mathbf{X}^T \mathbf{W} \mathbf{X}$ is the Fisher information matrix.
- With a canonical link the observed and expected Fisher information are equal

Fisher Scoring

Because we use the canonical model parameters the observed Fisher information matrix equals the expected Fisher information matrix $J(\beta) = I(\beta)$. Indeed, the observed Fisher information matrix is not depending on the observations, but only on the design and the variance of the data (via the weights).

Hence, Newton-Raphson is equivalent to Fisher scoring when the canonical link function is used.

Note, that the Fisher matrix, minus second derivative (or hessian) of the likelihood to the model parameters, is also the inverse of the variance covariance matrix of the model parameters. It is thus related to the precision.

Iteratively reweighted least squares

We can rewrite Newton Raphson or Fisher scoring as IRLS.

$$\begin{aligned}
\beta^{k+1} &= \beta^k + \left(\mathbf{X}^T\mathbf{W}\mathbf{X}\right)^{-1} S(\beta^k) \\
\beta^{k+1} &= \beta^k + \left(\mathbf{X}^T\mathbf{W}\mathbf{X}\right)^{-1} \mathbf{X}^T\mathbf{A}\left(\mathbf{Y} - \mu\right) \\
\beta^{k+1} &= \left(\mathbf{X}^T\mathbf{W}\mathbf{X}\right)^{-1} \mathbf{X}^T\mathbf{W}\mathbf{X}\beta^k + \left(X^T\mathbf{W}\mathbf{X}\right)^{-1} \mathbf{X}^T\mathbf{W}\frac{\partial\eta}{\partial\mu}\left(\mathbf{Y} - \mu\right) \\
\beta^{k+1} &= \left(\mathbf{X}^T\mathbf{W}\mathbf{X}\right)^{-1} \mathbf{X}^T\mathbf{W}\left[\mathbf{X}\beta^k + \frac{\partial\eta}{\partial\mu}\left(\mathbf{Y} - \mu\right)\right] \\
\beta^{k+1} &= \left(\mathbf{X}^T\mathbf{W}\mathbf{X}\right)^{-1} \mathbf{X}^T\mathbf{W}\mathbf{z}
\end{aligned}$$

with $\mathbf{z} = \left[\mathbf{X}\beta^k + \frac{\partial\eta}{\partial\mu}\left(\mathbf{Y} - \mu\right)\right]$

Each iteration is a **weighted least squares** fit of the working response $\mathbf{z}$ on $\mathbf{X}$, as in a linear model. The weights account for the mean-variance relation: observations with a smaller variance get more weight.

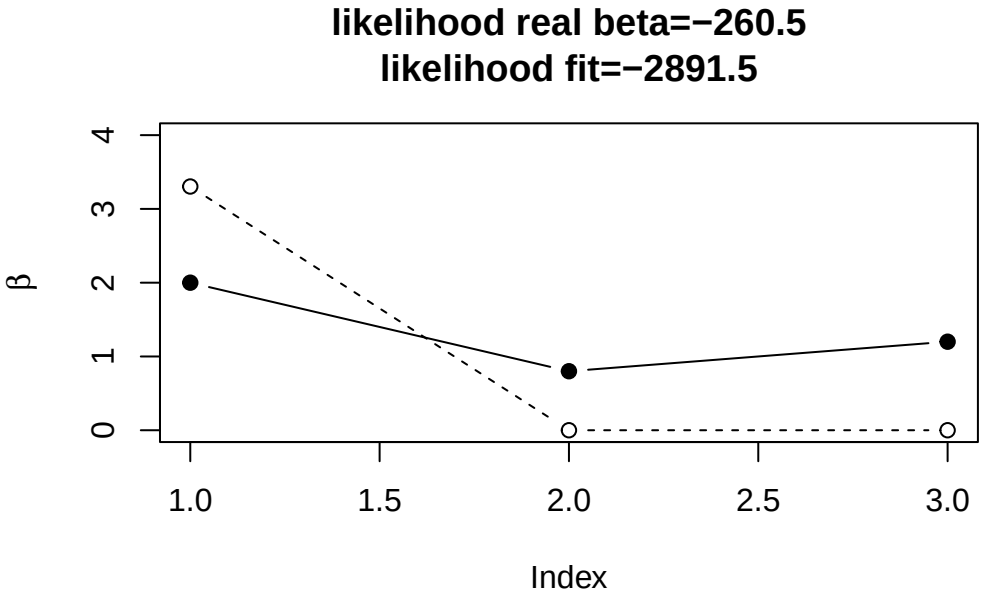
For the Poisson GLM with log link: $z_i = \eta_i + e^{-\eta_i}(y_i - e^{\eta_i})$ and $w_{ii} = e^{\eta_i}$.

IRLS in R

Initial Estimate

This is a very poor initial estimate used to illustrate the algorithm. Otherwise convergence for this simple example is way too quick

```r
iteration=0
betas<-c(log(mean(y)),0,0)
plot(betasTrue,
     ylab=expression(beta),
     ylim=c(0,4),
     pch=19,
     type="b",
     main=paste0("likelihood real beta=",
                 round(sum(dpois(y,exp(etaTrue),log=TRUE)),1),"\nlikelihood fit=", round(sum
)
lines(betas,type="b",lty=2 )
```



Iteratively Reweighted Least Squares

Pseudo Data

$$z_i = \eta_i + \frac{\partial \eta_i}{\partial \mu_i}(y_i - \mu_i)$$

$$z_i = \eta_i + e^{-\eta_i} y_i - 1$$

Weight Matrix?

$$[w_{ii}] = var_{y_i}^{-1} \left(\frac{\partial \mu}{\partial \eta}\right)^2$$

$$[w_{ii}] = e^{\eta_i}$$

Run Update Step Multiple Times

First 3 times (colors are black 0, red iteration 1, green iteration 2, blue iteration 3)

```
plot(betasTrue,ylab=expression(beta),ylim=c(0,4),pch=19,type="b")
lines(betas,type="b",lty=2)

cat("\nlikelihood TRUE=", round(sum(dpois(y,exp(X%*%betasTrue),log=TRUE)),1))
```

```
likelihood TRUE= -260.5
```

```
cat("\nlikelihood initial fit=", round(sum(dpois(y,exp(X%*%betas),log=TRUE)),1))
```

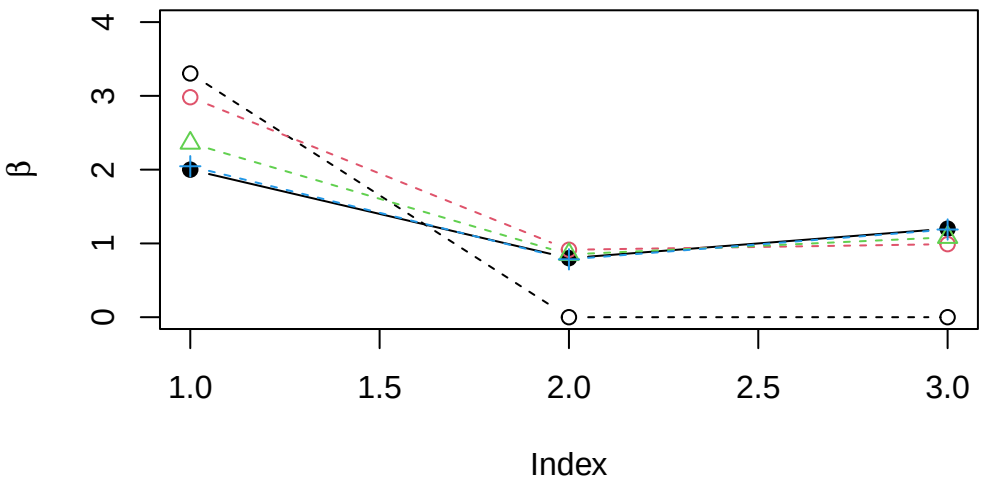
```
likelihood initial fit= -2891.5
```

```
#Calculate current eta
eta<-X%*%betas

iteration=0
for (i in 1:3)
{
#start IRLS UPDATE STEP
iteration=iteration+1
#calculate pseudo data based on current betas
z=eta+exp(-eta)*(y-exp(eta))
#calculate new weights: diagonal elements
w<-c(exp(eta))

#update betas
lmUpdate<-lm(z~-1+X,weight=w)
#eta<-X%*%betas
eta<-lmUpdate$fitted
```

```
betas<-lmUpdate$coef
lines(betas,type="b",col=iteration+1,pch=iteration,lty=2)
cat("\nlikelihood current fit=", round(sum(dpois(y,exp(X%*%betas),log=TRUE)),1))
}
```

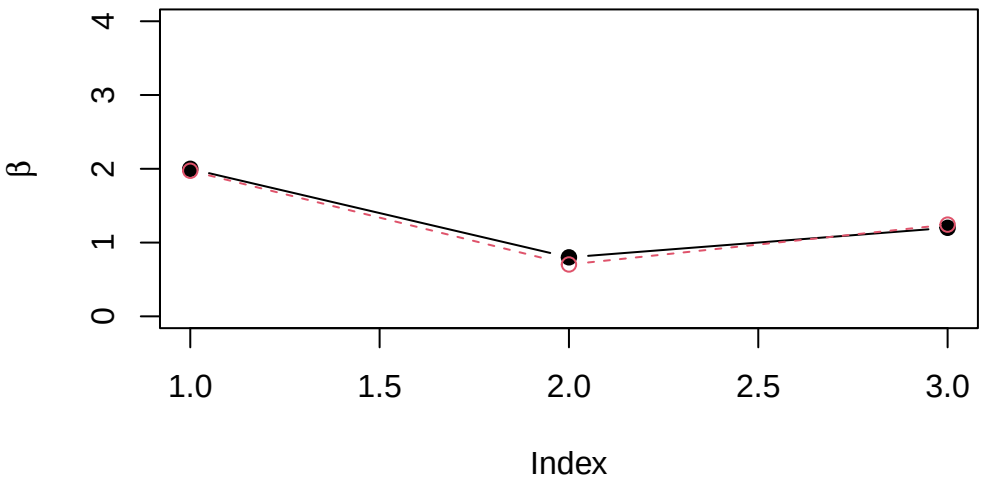



```
likelihood current fit= -1883.9
likelihood current fit= -431.8
likelihood current fit= -261.7
```

Comparison with GLM Function

Smarter Initialisation

```
z<-log(y+.5)
betas<-lm(z~-1+X)$coef
plot(betasTrue,ylab=expression(beta),ylim=c(0,4),pch=19,type="b")
lines(betas,col=2,type="b",lty=2)
```




```
#calculate current eta
eta<-X%*%betas

cat("\nlikelihood TRUE=", round(sum(dpois(y,exp(X%*%betasTrue),log=TRUE)),1))
```

```
likelihood TRUE= -260.5
```

```
cat("\nlikelihood initial fit=", round(sum(dpois(y,exp(X%*%betas),log=TRUE)),1))
```

```
likelihood initial fit= -263.8
```

Evaluation Stopping Criterion

- Residual deviance: Is 2 log of LR between best possible fit and current fit

$$LR = \frac{L_{\text{best}}}{L_{\text{current}}}$$

$$D = 2(\log L_{\text{best}} - \log L_{\text{current}})$$

$$D = 2(l_{\text{best}} - l_{\text{current}})$$

- Best fit: $\mu = y$
- Optimal poisson:

$$l_{\text{best}} = \sum [y_i \log(y_i) - y_i - \log(y_i!)]$$

- Current fit

$$l_{\text{current}} = \sum [y_i \eta_i - e^{\eta_i} - log(y_i!)]$$

- Deviance D:

$$D = 2 \sum [y_i log(y_i) - y_i \eta_i - (y_i - e^{\eta_i})]$$

- Problem to calculate it if y=0 but by apply l'Hopital's rule we know

$$\lim_{y_i \to 0} y_i \log(y_i) = 0$$

```
ylogy<-function(y)
{
return(ifelse(y==0,rep(0,length(y)),y*log(y)))
}

deviance<-2*sum(ylogy(y)-y*eta-(y-exp(eta)))

devianceOld<-1e30
```

Run Update Step until Convergence

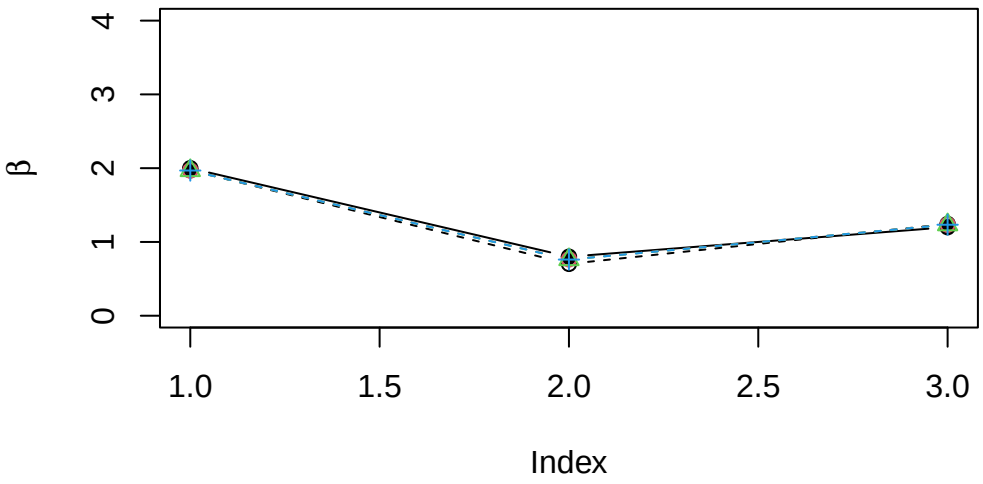
```
plot(betasTrue,ylab=expression(beta),ylim=c(0,4),pch=19,type="b")
lines(betas,type="b",lty=2)

tol<-1e-6
iteration=0
while(((devianceOld-deviance)/devianceOld)>tol)
{
#start IRLS UPDATE STEP
iteration=iteration+1
#calculate pseudo data based on current betas
z=eta+exp(-eta)*(y-exp(eta))
#calculate new weights: diagonal elements
w<-c(exp(eta))

#update betas
lmUpdate<-lm(z~-1+X,weight=w)
#eta<-X%*%betas
eta<-lmUpdate$fitted
```

```
betas<-lmUpdate$coef
lines(betas,type="b",col=iteration+1,pch=iteration,lty=2)

#criterion for convergence
devianceOld<-deviance
deviance<-2*sum(ylogy(y)-y*eta-(y-exp(eta)))
cat("iteration",iteration,"Deviance Old",devianceOld,"Deviance", deviance,"\n")
}
```




```
iteration 1 Deviance Old 129.1127 Deviance 114.3748
iteration 2 Deviance Old 114.3748 Deviance 114.3374
iteration 3 Deviance Old 114.3374 Deviance 114.3374
```

Variance β?

$$\Sigma_\beta = (\mathbf{X}^T\mathbf{W}\mathbf{X})^{-1}$$

```
varBeta=solve(t(X)%*%diag(w)%*%X)
```

Comparison with GLM fit

Use -1 because intercept is already in X

```
glmfit=glm(y~-1+X,family=poisson)
comp=data.frame(glmfit=c(glmfit$deviance,glmfit$coef,summary(glmfit)$coef[,2]),ourFit=c(devia
row.names(comp)=c("deviance",paste("beta",1:3,sep=""),paste("se",1:3,sep=""))
comp
```

```
               glmfit       ourFit
deviance 114.33739500 114.33739500
beta1      1.96805691   1.96805691
beta2      0.76136641   0.76136641
beta3      1.23330031   1.23330031
se1        0.03814382   0.03814378
se2        0.01891208   0.01891203
se3        0.02556665   0.02556659
```

Inference

Wald test

The maximum likelihood estimator is asymptotically normal:

$$\hat{\beta} \overset{a}{\sim} N\left[\beta, (\mathbf{X}^T\mathbf{W}\mathbf{X})^{-1}\right].$$

Under $H_0 : \mathbf{L}^T\beta = 0$,

$$\frac{L^T\hat{\beta}}{\widehat{\mathrm{SE}}_{L^T\hat{\beta}}} \xrightarrow{d} \mathcal{N}(0,1)$$

For c contrasts simultaneously, $\mathbf{L}$ is a $p \times c$ matrix and

Under $H_0 : \mathbf{L}\beta = \mathbf{0}$,

$$W = (\mathbf{L}\hat{\beta})^T \left(\mathbf{L}\widehat{\Sigma}_{\hat{\beta}}\mathbf{L}^T\right)^{-1} (\mathbf{L}\hat{\beta}) \xrightarrow{d} \chi^2_c$$

Deviance and likelihood ratio test

- The **deviance** compares the fit of the model with the saturated model, which fits every observation perfectly ($\hat{\mu}_i = y_i$):

$$D = 2\left[l(\mathbf{y}|\mathbf{y}) - l(\hat{\mu}|\mathbf{y})\right].$$

  It plays the role of the residual sum of squares in a linear model.

- The **likelihood ratio test** compares a full model with a nested reduced model, e.g. without the treatment effect:

$$H_0 : \text{LRT} = 2\left[l(\hat{\beta}_{\text{full}}) - l(\hat{\beta}_0)\right] \xrightarrow{d} \chi^2_c,$$

  with c the number of parameters that are dropped.

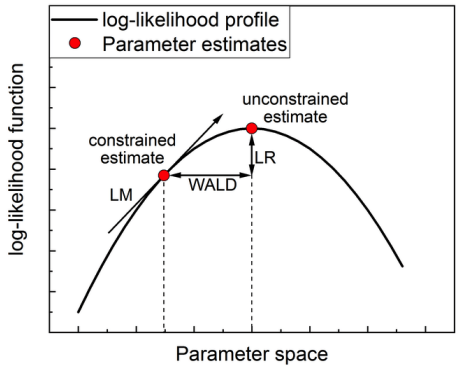


Figure 2: The Wald test looks at the distance between the estimate and the null value on the parameter scale, the likelihood ratio test at the difference in log-likelihood.

These results are asymptotic.

- With the typical 3 to 5 biological replicates per group in bulk RNA-seq, the large sample theory does not hold.
- Moreover, the dispersion of the NB distribution has to be estimated, which adds uncertainty that the χ^2 tests ignore.
- Quasi-likelihood F-tests address both issues.

Differential expression with edgeR

The negative binomial GLM

$$\left\{\begin{array}{lcl} Y_{gi} & \sim & NB(\mu_{gi}, \phi_g) \\ \log \mu_{gi} & = & \mathbf{x}_i^T \beta_g + \log N_i^{\text{eff}} \end{array}\right. \quad \text{with} \quad \text{Var}[Y_{gi}] = \mu_{gi} + \phi_g \mu_{gi}^2.$$

- For a known dispersion ϕ_g, the NB distribution is a member of the exponential family and the model is fitted with IRLS.
- The original edgeR pipeline (McCarthy et al. 2012) estimates the dispersions with an adjusted profile likelihood (Cox-Reid), which corrects for the estimation of the mean parameters, similar to REML in mixed models.
- Gene-wise dispersions are very unstable with few replicates. They are shrunken towards a trend in the average expression with empirical Bayes, and the genes are then tested with likelihood ratio tests.
- The uncertainty in the estimated dispersions is ignored by these tests.

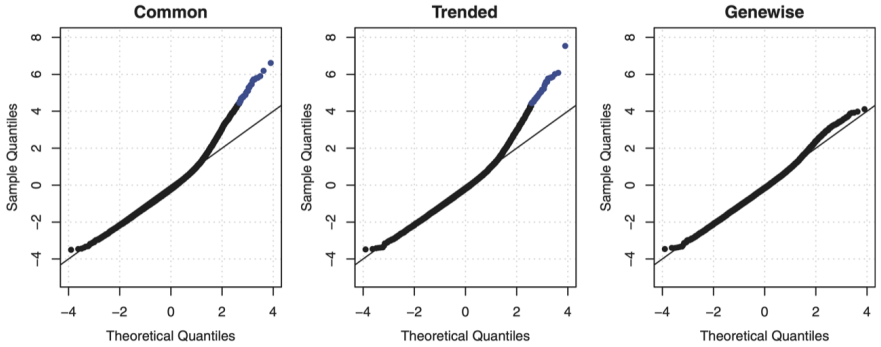


Figure 3: QQ-plots of goodness of fit statistics with a common, trended or empirical Bayes (tagwise) dispersion (from McCarthy et al., 2012).

Quasi-likelihood

In quasi-likelihood we no longer specify the full distribution, but only the mean and the variance:

$$\left\{\begin{array}{lcl} E[Y_i|\mathbf{x}_i] & = & \mu_i \\ g(\mu_i) & = & \mathbf{x}_i^T \beta \\ \text{Var}[Y_i] & = & \sigma^2 V(\mu_i) \end{array}\right.$$

- $V(\mu_i)$ is the variance function of a known distribution, e.g. $V(\mu) = \mu$ (quasi-Poisson) or $V(\mu) = \mu + \phi\mu^2$ (quasi-NB).
- σ^2 is the **quasi-likelihood dispersion**, which allows for more ($\sigma^2 > 1$) or less ($\sigma^2 < 1$) variability than the original distribution.

Estimation

The estimating equations become

$$U(\beta) = \frac{1}{\sigma^2}\mathbf{X}^T\mathbf{A}\,(\mathbf{y} - \mu) = \mathbf{0} \quad \Leftrightarrow \quad \mathbf{X}^T\mathbf{A}\,(\mathbf{y} - \mu) = \mathbf{0}.$$

- The estimators of the mean parameters are identical to those of the original GLM: we can use the same IRLS algorithm.
- The variance-covariance matrix is inflated: $\Sigma_{\hat{\beta}} = \sigma^2(\mathbf{X}^T\mathbf{W}\mathbf{X})^{-1}$.
- σ^2 is estimated from the residuals, e.g. based on the deviance:

$$\hat{\sigma}^2 = \frac{D}{n-p},$$

  the GLM counterpart of the residual variance $\hat{\sigma}^2 = \text{SSE}/(n-p)$ in a linear model.

Inference

- t-test: $t = \frac{\mathbf{L}^T\hat{\beta}}{\widehat{\text{SE}}^{QL}} \overset{a}{\sim} t_{n-p}$ under H_0, with $\text{SE}^{QL} = \hat{\sigma}\,\text{SE}^{ML}$.
- F-test: $F = \frac{(D_0 - D_{\text{full}})/c}{\hat{\sigma}^2} \overset{a}{\sim} F_{c,n-p}$ under H_0.

The t and F distributions account for the uncertainty in $\hat{\sigma}^2$, as in linear models.

Example: quasi-Poisson

```
counts <- c(18, 17, 15, 20, 10, 20, 25, 13, 12)
outcome <- gl(3, 1, 9)
treatment <- gl(3, 3)
glmPois <- glm(counts ~ outcome + treatment, family = poisson())
glmQPois <- glm(counts ~ outcome + treatment, family = quasipoisson())
```

The quasi-Poisson fit gives the same estimates, but its standard errors are inflated with $\hat{\sigma}$. `glm` estimates σ^2 with the Pearson residuals, $\hat{\sigma}^2 = \sum_i \frac{(y_i - \hat{\mu}_i)^2}{V(\hat{\mu}_i)}/(n-p)$.

```r
sigma2 <- sum(residuals(glmPois, type = "pearson")^2) / df.residual(glmPois)
sigma2
```

```
[1] 1.2933
```

```r
cbind(
  estimate = coef(glmPois),
  sePois = summary(glmPois)$coef[, 2],
  seQPois = summary(glmQPois)$coef[, 2],
  sePoisTimesSigma = summary(glmPois)$coef[, 2] * sqrt(sigma2)
)
```

```
                estimate    sePois   seQPois sePoisTimesSigma
(Intercept)  3.044522e+00 0.1708987 0.1943517        0.1943517
outcome2    -4.542553e-01 0.2021708 0.2299154        0.2299154
outcome3    -2.929871e-01 0.1927423 0.2191931        0.2191931
treatment2   1.397701e-16 0.2000000 0.2274467        0.2274467
treatment3  -2.415657e-16 0.2000000 0.2274467        0.2274467
```

The F-test for the outcome effect equals the LRT divided by its degrees of freedom and $\hat{\sigma}^2$:

```r
glmQPois0 <- glm(counts ~ treatment, family = quasipoisson())
anova(glmQPois0, glmQPois, test = "F")
```

```
Analysis of Deviance Table

Model 1: counts ~ treatment
Model 2: counts ~ outcome + treatment
  Resid. Df Resid. Dev Df Deviance      F Pr(>F)
1         6    10.5814
2         4     5.1291  2   5.4523 2.1079  0.237
```

```r
(deviance(glmQPois0) - deviance(glmQPois)) / 2 / sigma2
```

```
[1] 2.107903
```

The edgeR quasi-likelihood pipeline

edgeR (Lund et al. 2012; Chen et al. 2025) uses a quasi-NB model with a gene-specific QL dispersion:

$$\mathrm{Var}[Y_{gi}] = \sigma_g^2 \left(\mu_{gi} + \phi \mu_{gi}^2\right).$$

We illustrate it on the macrophage data of the introduction: 6 donors, with IFNγ priming and *Salmonella* infection in a 2 × 2 factorial design (Alasoo et al. 2018).

```
macDir <- system.file("extdata", package = "macrophage")
coldata <- read.csv(file.path(macDir, "coldata.csv")) |>
  transmute(
    names,
    files = file.path(macDir, "quants", names, "quant.sf.gz"),
    donor = factor(line_id),
    condition = factor(condition_name, levels = c("naive", "IFNg", "SL1344", "IFNg_SL1344"))
    ifng = factor(ifelse(grepl("IFNg", condition_name), "yes", "no")),
    salmonella = factor(ifelse(grepl("SL1344", condition_name), "yes", "no"))
  )
st <- tximeta(coldata, dropInfReps = TRUE)
sg <- summarizeToGene(st)
sg <- addIds(sg, "SYMBOL", gene = TRUE)

dge <- DGEListFromTximeta(sg)
dge$genes <- dge$genes[, c("SYMBOL", "Max2MinLength")]
design <- model.matrix(~ donor + ifng * salmonella, data = colData(sg))
keep <- filterByExpr(dge, design)
dge <- dge[keep, , keep.lib.sizes = FALSE]
dge <- normLibSizes(dge)
```

As in the introduction, `DGEListFromTximeta` stores the centred log average transcript lengths in `dge$offset.prior`. `getOffset` returns the offsets that edgeR uses in all models: the log effective library sizes plus these length offsets.

```
getOffset(dge)[1:3, 1:4]
```

```
                  SAMEA103885102 SAMEA103885347 SAMEA103885043 SAMEA103885392
ENSG00000000003.14       17.83113       17.81073       17.51715       17.36446
ENSG00000000419.12       17.83579       17.80954       17.53832       17.37225
ENSG00000000457.13       17.98175       17.84456       17.58191       17.30384
```

Step 1: a global NB dispersion

- `glmQLFit` no longer needs gene-wise NB dispersions.
- It estimates a single NB dispersion ϕ from the most highly expressed genes, for which the Poisson noise is negligible.

```
fit <- glmQLFit(dge, design)
fit$dispersion
```

```
[1] 0.08474179
```

```
fit$top.proportion
```

```
[1] 0.08323239
```

- $\phi = 0.085$ corresponds to a biological coefficient of variation of 29%.
- All gene-specific variability is captured by the QL dispersions σ_g^2.
- Older edgeR analyses first estimate trended NB dispersions with `estimateDisp` and use the classical QL theory (`glmQLFit(..., legacy = TRUE)`). For bulk RNA-seq with moderate counts, both pipelines give very similar results.

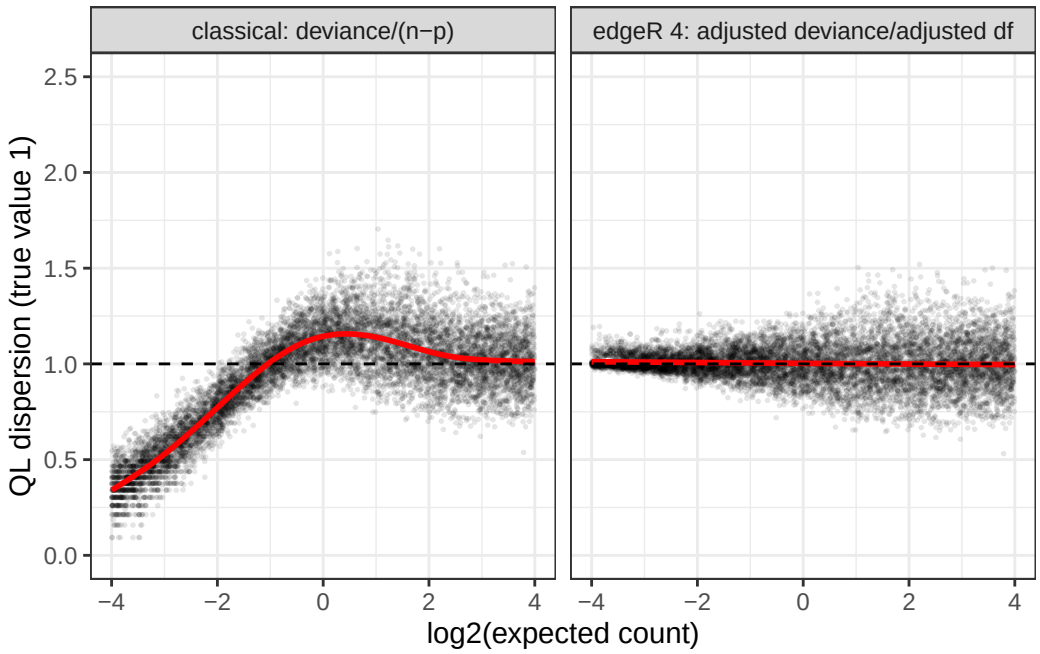
Step 2: QL dispersions with bias-adjusted degrees of freedom

- The QL dispersion is estimated as $\hat{\sigma}_g^2 = D_g/\mathrm{df}_g$.
- Classical QL theory treats the deviance as a scaled χ^2_{n-p}. This works well for large counts but not for small counts, where the QL dispersions are biased (Chen et al. 2025).
- edgeR 4 corrects both the deviance and the residual df for the count size, which gives nearly unbiased QL dispersions.
- As a consequence, the residual df are fractional and gene-specific. They are used in the empirical Bayes step and in the F-test.

The plot below reproduces Fig. 1 of Chen et al. (2025): Poisson counts for 10 000 genes and 100 samples. Because the counts are simulated from the same variance model that is used in the fit (here Poisson, i.e. NB dispersion 0), the true QL dispersion equals 1 for every gene.

```
set.seed(1)
mu <- 2^runif(10000, -4, 4)
ySim <- matrix(rpois(10000 * 100, mu), 10000)
designSim <- matrix(1, 100, 1)
fitClassic <- glmQLFit(ySim, designSim, dispersion = 0, legacy = TRUE)
fitAdj <- glmQLFit(ySim, designSim, dispersion = 0, legacy = FALSE)
```

```
tibble(log2mu = log2(mu),
       `classical: deviance/(n-p)` = fitClassic$deviance / fitClassic$df.residual,
       `edgeR 4: adjusted deviance/adjusted df` = fitAdj$deviance.adj / fitAdj$df.residual.ad
  pivot_longer(-log2mu, names_to = "method", values_to = "QLdisp") |>
  ggplot(aes(log2mu, QLdisp)) +
  geom_point(alpha = 0.1, size = 0.3) +
  geom_smooth(col = "red", se = FALSE) +
  geom_hline(yintercept = 1, lty = 2) +
  facet_wrap(~method) +
  coord_cartesian(ylim = c(0, 2.5)) +
  xlab("log2(expected count)") + ylab("QL dispersion (true value 1)") +
  theme_bw()
```



- The classical estimator underestimates the QL dispersion for expected counts below about 0.5 and overestimates it around 1–2.
- The adjusted estimator is unbiased for all count sizes and is also more precise at low counts.

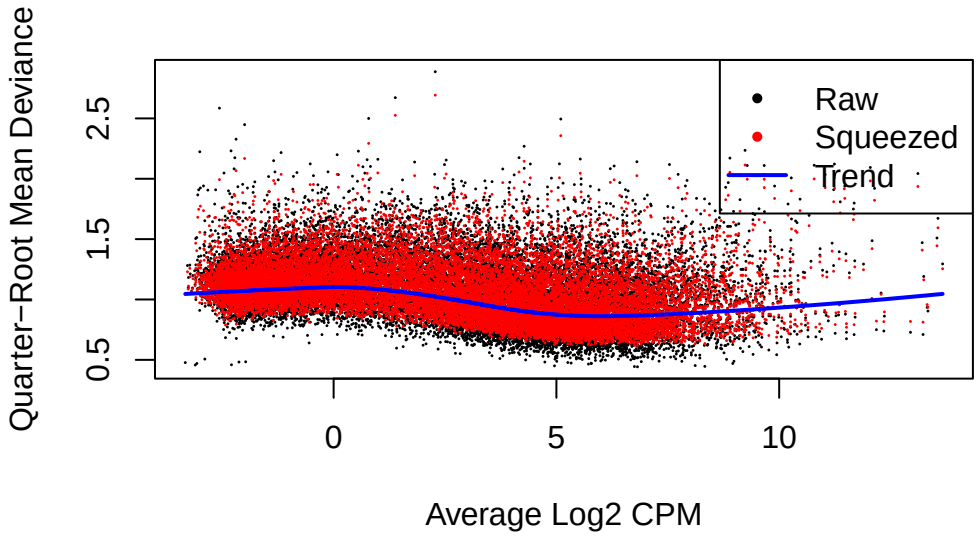
Step 3: empirical Bayes

The QL dispersions are shrunken towards a trend in the average expression with `squeezeVar`, exactly as the residual variances in limma and msqrob2:

$$\tilde{\sigma}_g^2 = \frac{d_0 s_0^2 + d_g \hat{\sigma}_g^2}{d_0 + d_g},$$

with s_0^2 the trend, d_0 the prior degrees of freedom and d_g the (adjusted) residual degrees of freedom of gene g.

```
plotQLDisp(fit)
```




```
fit$df.prior
```

```
[1] 3.903063
```

Step 4: moderated F-tests

$$F_g = \frac{(D_{0,g} - D_{\text{full,g}})/c}{\tilde{\sigma}_g^2} \overset{a}{\sim} F_{c,d_0+d_g} \text{ under } H_0.$$

For a single parameter ($c = 1$), e.g. the IFNγ effect in uninfected cells, the F-test is equivalent to a moderated t-test:

```
qlf <- glmQLFTest(fit, coef = "ifngyes")
topTags(qlf, n = 5)
```

```
Coefficient:  ifngyes
                     SYMBOL Max2MinLength    logFC   logCPM        F
ENSG00000174944.8   P2RY14      1.473634 9.813128 5.197276 453.4029
ENSG00000111181.12 SLC6A12      2.430672 4.717016 4.299140 555.5078
ENSG00000125347.13    IRF1      1.110418 5.573370 9.630721 504.7893
ENSG00000234518.2     <NA>      1.120846 8.120867 3.013158 286.0034
ENSG00000137496.17  IL18BP      1.367702 4.068700 7.470237 469.9069
                          PValue          FDR
ENSG00000174944.8  5.449938e-18 1.047696e-13
ENSG00000111181.12 1.795615e-15 1.725945e-11
ENSG00000125347.13 4.256966e-15 2.727864e-11
ENSG00000234518.2  7.847271e-15 3.144090e-11
ENSG00000137496.17 8.177514e-15 3.144090e-11
```

```
summary(decideTests(qlf))
```

```
       ifngyes
Down      2648
NotSig   13475
Up        3101
```

With several parameters we test whether any of them differs from zero, e.g. whether a gene responds to any of the treatments ($c = 3$):

```
qlfAny <- glmQLFTest(fit, coef = c("ifngyes", "salmonellayes", "ifngyes:salmonellayes"))
topTags(qlfAny, n = 5)
```

```
Coefficient:  ifngyes salmonellayes ifngyes:salmonellayes
                      SYMBOL Max2MinLength logFC.ifngyes logFC.salmonellayes
ENSG00000255521.1   CD44-DT      1.818456     -1.320271           6.8065895
ENSG00000171989.5   LDHAL6B      1.037796      2.259354           0.7179383
ENSG00000182393.2     IFNL1      1.127983      1.191166           7.7333750
ENSG00000279965.1      <NA>      1.040066      2.733704           2.0793137
ENSG00000185433.8 LINC00158      2.222640      3.542199           7.5644149
                  logFC.ifngyes.salmonellayes    logCPM        F       PValue
ENSG00000255521.1                    4.601783 2.9315851 627.3802 1.933860e-25
ENSG00000171989.5                    5.626021 0.5407264 329.7240 7.550937e-25
```

```
ENSG00000182393.2                   5.840472 7.6971616 480.5455 1.181854e-24
ENSG00000279965.1                   6.388032 2.5174241 340.7399 3.563758e-23
ENSG00000185433.8                  -1.280528 2.3879599 374.8519 6.616562e-22
                            FDR
ENSG00000255521.1 3.717653e-21
ENSG00000171989.5 7.257961e-21
ENSG00000182393.2 7.573323e-21
ENSG00000279965.1 1.712742e-19
ENSG00000185433.8 2.543936e-18
```

```
summary(decideTests(qlfAny))
```

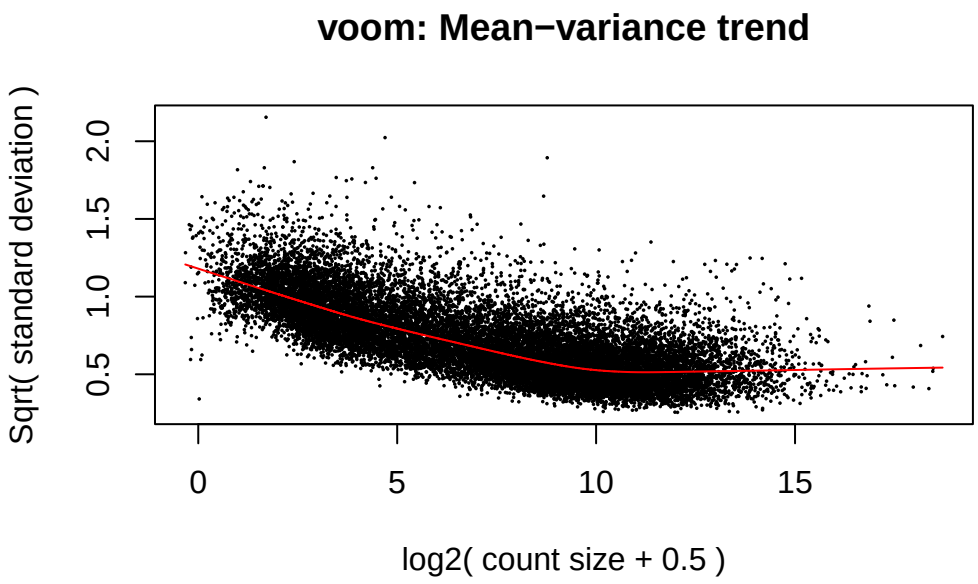
```
       ifngyes:salmonellayes-salmonellayes-ifngyes
NotSig                                        3971
Sig                                          15253
```

limma-voom

An alternative is to model log-transformed counts with linear models (Law et al. 2014).

- voom computes log2 CPM values with a prior count of 0.5, so that zero counts can be log-transformed: $y_{gi} = \log_2\{(Y_{gi} + 0.5)/(N_{gi} + 1) \times 10^6\}$, with N_{gi} the effective library size, which includes the TMM normalisation factors and the length offsets of tximeta.
- The log2 CPM values are still heteroscedastic: low counts are more variable.
- voom estimates the mean-variance trend of the log2 CPM values and converts it into a **precision weight** for every observation.
- The weighted log2 CPM values are then analysed with the limma pipeline: linear models, empirical Bayes moderation with `squeezeVar` and moderated t-tests.

```
v <- voom(dge, design, plot = TRUE)
```




```
fitVoom <- lmFit(v, design) |>
  eBayes()
summary(decideTests(fitVoom))[, "ifngyes"]
```

```
  Down NotSig     Up
  2664  13375   3185
```

- voom is fast and has access to the full limma toolbox: e.g. random effects via `duplicateCorrelation`, array weights and gene set tests.
- It works well when the library sizes are not too different and there are sufficient replicates.
- It becomes problematic for small counts. After the log transformation, counts of 0, 1 or 2 take only a few discrete values that depend on the prior count, and their distribution is far from normal. The mean-variance trend is also estimated less reliably at low counts, so these observations get less appropriate weights.
- edgeR 4 models the counts directly and corrects the deviance and degrees of freedom for small counts, which gives better error control when the data are dominated by small counts (Chen et al. 2025).

DESeq2

DESeq2 (Love et al. 2014) also uses an NB GLM, but

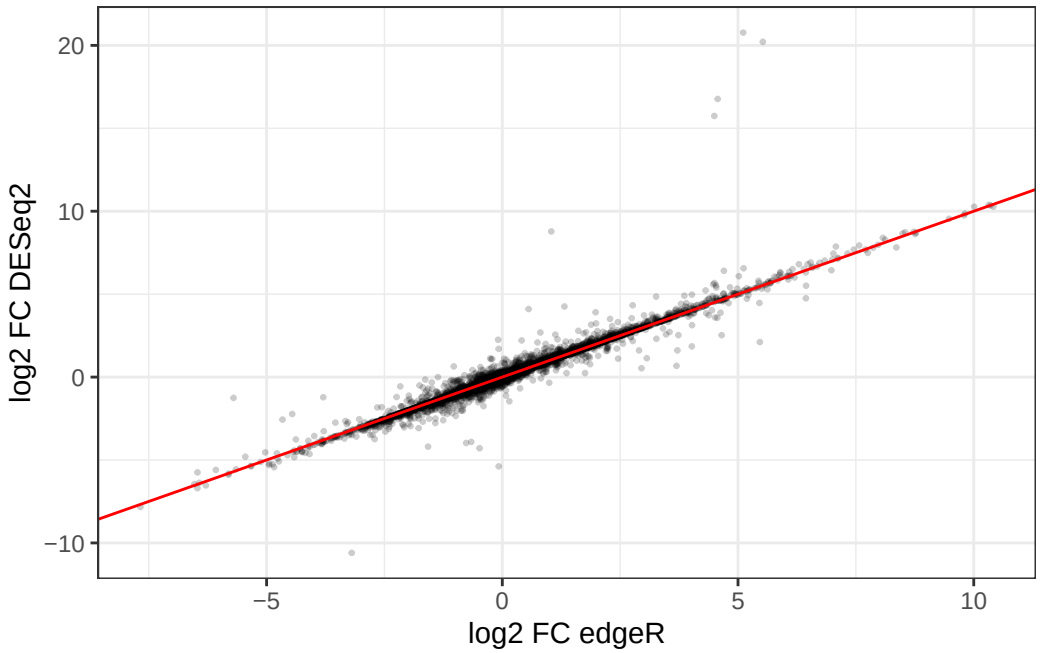
- normalises with the median of ratios method (as in proteomics),
- estimates gene-wise dispersions ϕ_g and shrinks them towards a trend with empirical Bayes,
- uses Wald tests (or LRTs) based on the normal distribution,
- optionally shrinks the log fold changes (e.g. with apeglm) for ranking and visualisation.

`DESeqDataSet` takes the tximeta object directly. It rounds the estimated counts and, like `DGEListFromTximeta` for edgeR, uses the average transcript lengths as additional offsets. Both analyses thus correct for transcript length.

```
dds <- DESeqDataSet(sg, design = ~ donor + ifng * salmonella)
dds <- dds[rownames(dge), ]
dds <- DESeq(dds, quiet = TRUE)
resDESeq <- results(dds, name = "ifng_yes_vs_no")
summary(resDESeq, alpha = 0.05)
```

```
out of 19224 with nonzero total read count
adjusted p-value < 0.05
LFC > 0 (up)       : 3244, 17%
LFC < 0 (down)     : 2930, 15%
outliers [1]       : 0, 0%
low counts [2]     : 0, 0%
(mean count < 1)
[1] see 'cooksCutoff' argument of ?results
[2] see 'independentFiltering' argument of ?results
```

```
tibble(
  edgeR = qlf$table$logFC,
  DESeq2 = resDESeq$log2FoldChange
) |>
  ggplot(aes(edgeR, DESeq2)) +
  geom_point(alpha = 0.2, size = 0.5) +
  geom_abline(intercept = 0, slope = 1, col = "red") +
  xlab("log2 FC edgeR") +
  ylab("log2 FC DESeq2") +
  theme_bw()
```



- The log fold changes agree well.
- The Wald tests of DESeq2 do not account for the uncertainty in the dispersion estimates and can be somewhat liberal with few replicates.

Normalisation

Normalisation corrects for technical differences between samples:

- **Sequencing depth**: a sample that is sequenced deeper has higher counts for all genes.
- **RNA composition**: if a few genes are very highly expressed in one sample, they take up a large fraction of the reads, and all other genes get lower counts.
- Other technical effects, e.g. GC content or transcript length, can also be accounted for.

We illustrate it on the parathyroid data. We first sum the counts of the technical replicates.

```
bfc <- BiocFileCache()
paraTar <- bfcrpath(bfc, "https://bioconductor.org/packages/3.21/data/experiment/src/contrib/
untar(paraTar, files = "parathyroidSE/data/parathyroidGenesSE.RData", exdir = tempdir())
load(file.path(tempdir(), "parathyroidSE/data/parathyroidGenesSE.RData"))

paraCounts <- t(rowsum(t(assay(parathyroidGenesSE)), colData(parathyroidGenesSE)$experiment))
paraSamples <- colData(parathyroidGenesSE) |>
```

```
  as.data.frame() |>
  distinct(experiment, .keep_all = TRUE) |>
  dplyr::select(experiment, patient, treatment, time)
paraCounts <- paraCounts[, paraSamples$experiment]
dim(paraCounts)
```

```
[1] 63193    23
```

We compare the four control samples at 48 hours in MD plots: the difference M in log2 count against the average log2 count A, for every pair of samples.

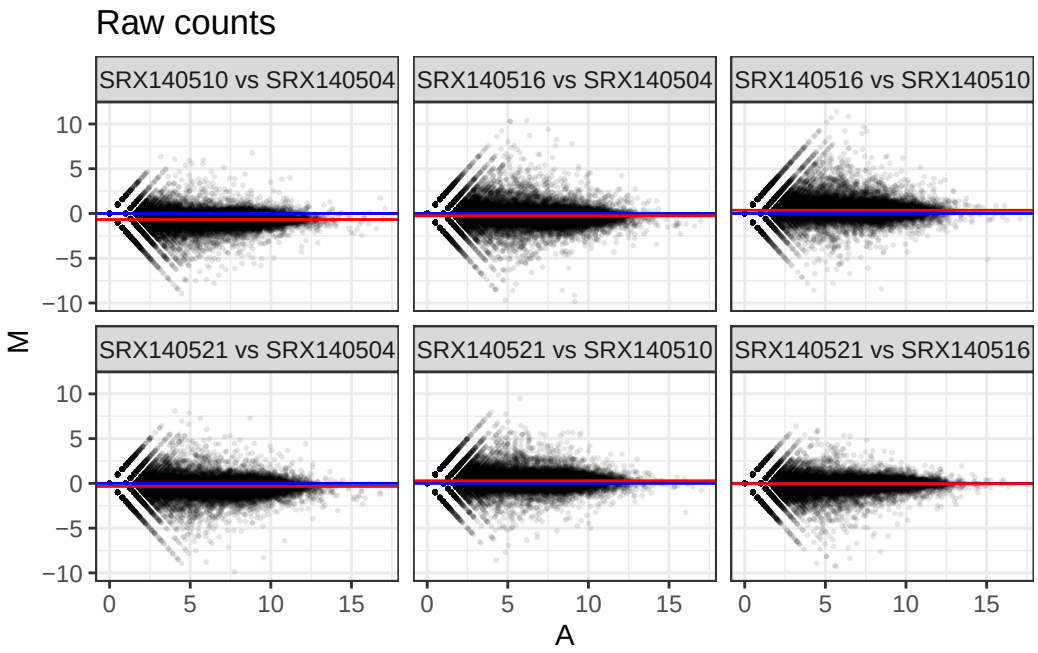
```
mdPairs <- function(counts, libSize) {
  pairs <- combn(ncol(counts), 2)
  map(seq_len(ncol(pairs)), function(k) {
    i <- pairs[1, k]
    j <- pairs[2, k]
    use <- counts[, i] > 0 & counts[, j] > 0
    tibble(
      pair = paste(colnames(counts)[j], "vs", colnames(counts)[i]),
      A = (log2(counts[use, i]) + log2(counts[use, j])) / 2,
      M = log2(counts[use, j] / libSize[j]) - log2(counts[use, i] / libSize[i])
    )
  }) |>
    bind_rows()
}

ctrl48 <- paraSamples$treatment == "Control" & paraSamples$time == "48h"
ctrlCounts <- paraCounts[, ctrl48]
colSums(ctrlCounts) / 1e6
```

```
SRX140504 SRX140510 SRX140516 SRX140521
10.827109  6.844144  8.064268  7.701432
```

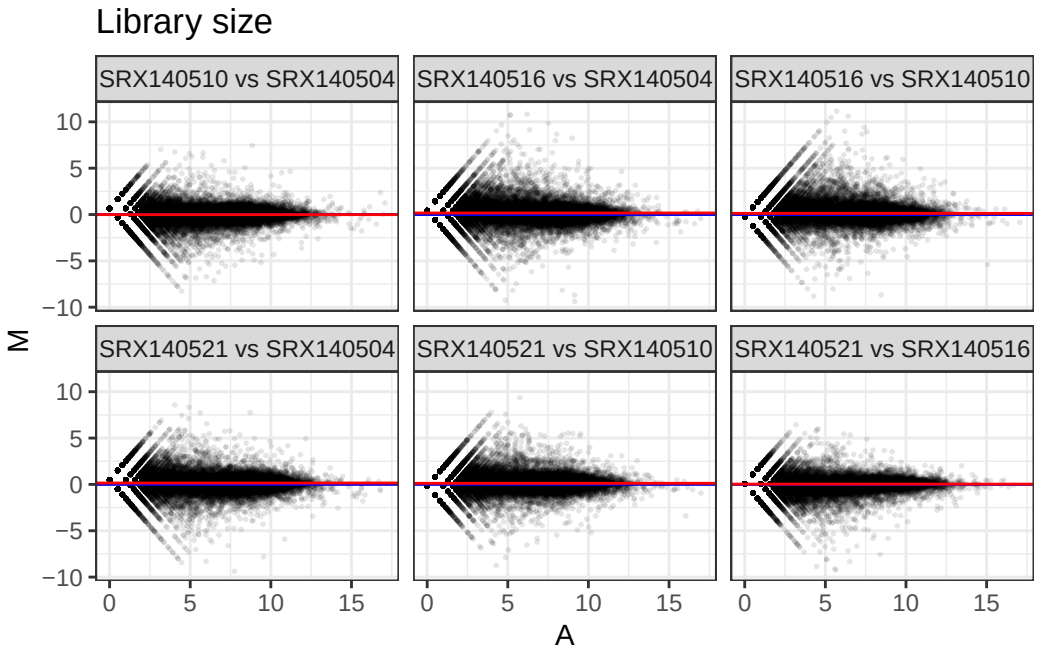
```
mdPlot <- function(mdData) {
  mdData |>
    group_by(pair) |>
    mutate(median = median(M)) |>
    ggplot(aes(A, M)) +
    geom_point(alpha = 0.1, size = 0.3) +
    geom_hline(yintercept = 0, col = "blue") +
```

```
    geom_hline(aes(yintercept = median), col = "red") +
    facet_wrap(~ pair) +
    theme_bw()
}
mdPlot(mdPairs(ctrlCounts, rep(1, ncol(ctrlCounts)))) +
  ggtitle("Raw counts")
```



The log fold changes between replicates are not centred around zero (blue line). Dividing by the library size, i.e. using the log library size as offset, removes most of the bias:

```
mdPlot(mdPairs(ctrlCounts, colSums(ctrlCounts))) +
  ggtitle("Library size")
```



TMM (edgeR)

The trimmed mean of M-values (TMM) method (Robinson and Oshlack 2010) assumes that most genes are not differentially expressed.

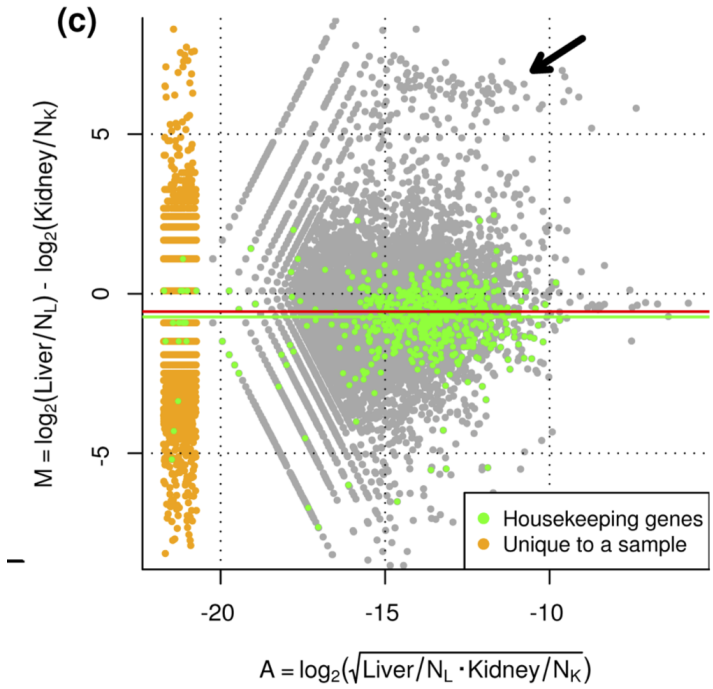


Figure 4: Composition bias: a few highly expressed genes in one sample take up a large fraction of the reads (from Robinson and Oshlack, 2010).

For every sample i, a normalisation factor F_i is computed with respect to a reference sample r:

$$\log_2 F_i = \frac{\sum_{g \in G^*} w_{gi} M_{gi}}{\sum_{g \in G^*} w_{gi}}, \quad M_{gi} = \log_2 \frac{Y_{gi}/N_i}{Y_{gr}/N_r}, \quad w_{gi} = \left(\frac{N_i - Y_{gi}}{N_i Y_{gi}} + \frac{N_r - Y_{gr}}{N_r Y_{gr}} \right)^{-1},$$

- G^*: the genes that remain after trimming the 30% most extreme M values and the 5% most extreme average expression values,
- w_{gi}: precision weights, the inverse of the approximate variance of M_{gi}, so that low counts get less weight.

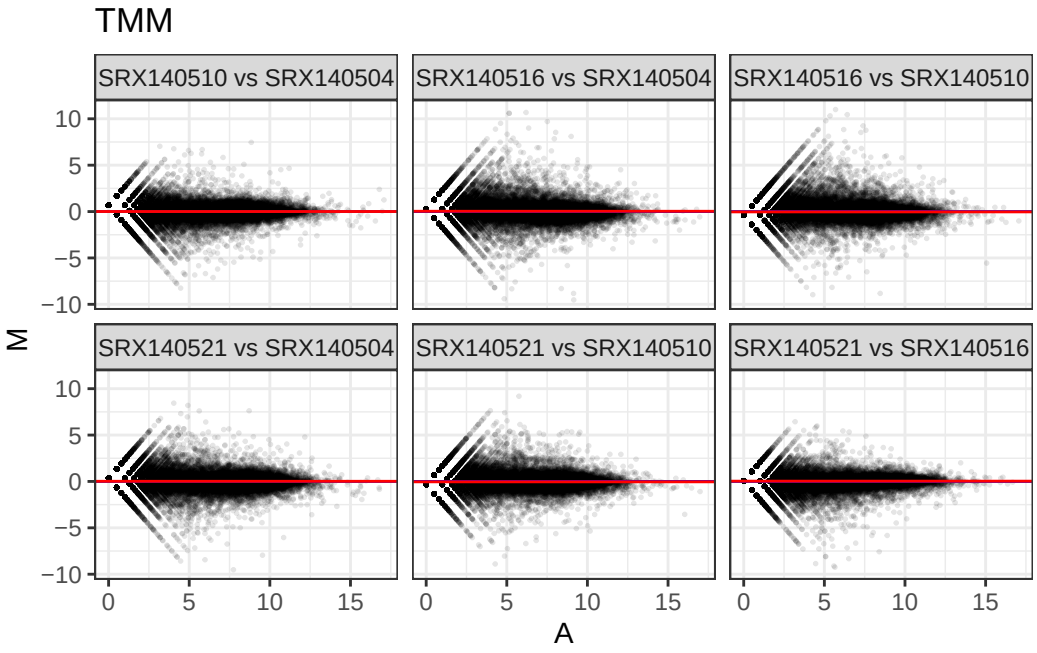
The effective library size $N_i^{\text{eff}} = N_i F_i$ is used as offset.

```
dgePara <- DGEList(ctrlCounts) |>
  normLibSizes()
dgePara$samples
```

```
       group lib.size norm.factors
```

```
SRX140504      1 10827109    0.9568210
SRX140510      1  6844144    0.9443689
SRX140516      1  8064268    1.0511507
SRX140521      1  7701432    1.0528406
```

```
effLibSize <- dgePara$samples$lib.size * dgePara$samples$norm.factors
mdPlot(mdPairs(ctrlCounts, effLibSize)) +
  ggtitle("TMM")
```



Median of ratios (DESeq2)

The median of ratios method (Love et al. 2014) that we used in proteomics computes a size factor for each sample with respect to a pseudo-reference sample, the geometric mean of the counts over all samples:

$$s_i = \text{median}_{(g:Y_g^*>0)} \frac{Y_{gi}}{Y_g^*}, \quad Y_g^* = \sqrt[n]{\prod_{i=1}^{n} Y_{gi}}.$$

```r
ddsPara <- DESeqDataSetFromMatrix(ctrlCounts, colData = DataFrame(sample = colnames(ctrlCount
ddsPara <- estimateSizeFactors(ddsPara)
tibble(
  sample = colnames(ctrlCounts),
  TMM = effLibSize / exp(mean(log(effLibSize))),
  MoR = sizeFactors(ddsPara) / exp(mean(log(sizeFactors(ddsPara))))
)
```

```
# A tibble: 4 x 3
  sample      TMM   MoR
  <chr>     <dbl> <dbl>
1 SRX140504 1.26  1.27
2 SRX140510 0.785 0.790
3 SRX140516 1.03  1.02
4 SRX140521 0.984 0.979
```

After rescaling both to a geometric mean of one, both methods give very similar normalisation factors.

Independent filtering

- Lowly expressed genes have little power to be found DE, but they count in the multiple testing correction.
- Removing them before testing can increase the number of discoveries at the same FDR.
- But we then only test the genes that pass the filter. The FDR is only controlled if the filter does not change the distribution of the test statistic of the null genes (Bourgon et al. 2010).

Filtering is valid if the filter statistic is independent of the test statistic under the null hypothesis.

- For every gene we compute two statistics: a filter statistic S_F and a test statistic S_T.
- Only genes that pass the filter are tested, and the p-values are adjusted for the genes that passed the filter only.
- The second stage is thus conditional on the first. The FDR is only controlled if the conditional null distribution of S_T for the genes that pass the filter equals its unconditional null distribution.

Let's build some intuition with simulated data.

- `makeExampleDESeqDataSet` simulates negative binomial counts for 1000 genes in two groups of 6 samples, without any differential expression: all genes are null genes.
- We test every gene with a two-sample t-test.

```
set.seed(24)
dds <- makeExampleDESeqDataSet()
simCounts <- counts(dds)
group <- dds$condition
testStat <- apply(simCounts, 1, function(x) t.test(x ~ group, var.equal = TRUE)$statistic)
```

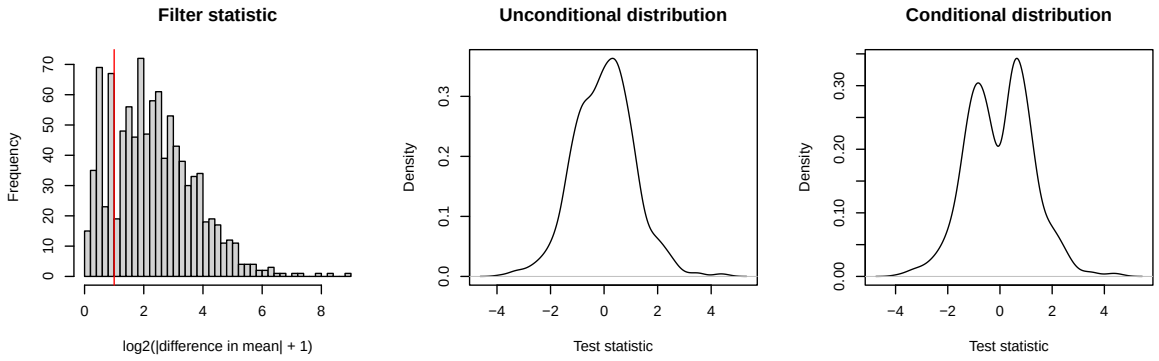
A dependent filter statistic

We filter on the absolute difference in mean between the groups, which depends on the group labels. The red line in the histogram of the filter statistic (on the log scale) gives the cutoff.

```
filterStatEffectSize <- abs(rowMeans(simCounts[, group == "A"]) - rowMeans(simCounts[, group
keepEffectSize <- filterStatEffectSize > 1
mean(keepEffectSize)
```

```
[1] 0.792
```

```
par(mfrow = c(1, 3))
hist(log2(filterStatEffectSize + 1), breaks = 40, main = "Filter statistic", xlab = "log2(|di
abline(v = log2(1 + 1), col = "red")
plot(density(testStat, na.rm = TRUE), xlab = "Test statistic", main = "Unconditional distribu
plot(density(testStat[keepEffectSize], na.rm = TRUE), xlab = "Test statistic", main = "Condit
```



- The genes that pass the filter have large test statistics: the conditional null distribution has lost its centre and is very different from the unconditional one.

- The p-values of the genes that pass the filter are therefore too small and the FDR is no longer controlled.

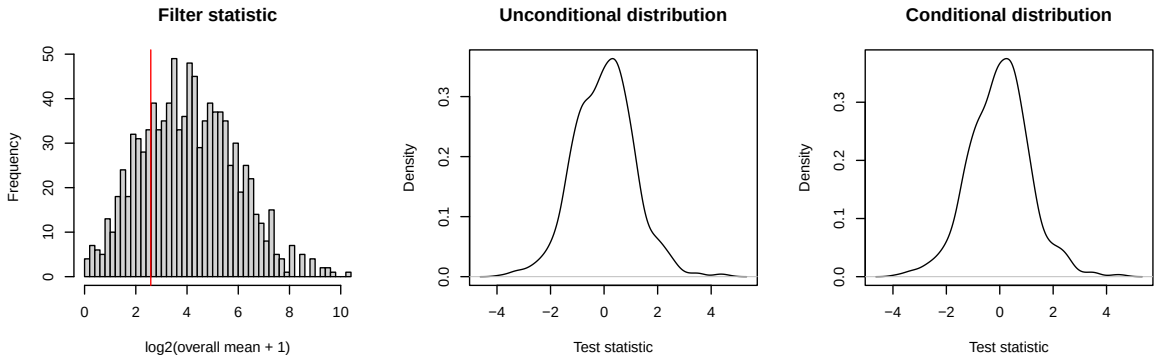
An independent filter statistic

We filter on the overall mean, irrespective of the group labels, and remove a similar fraction of the genes.

```
filterStatGlobalMean <- rowMeans(simCounts)
keepGlobalMean <- filterStatGlobalMean > 5
mean(keepGlobalMean)
```

```
[1] 0.771
```

```
par(mfrow = c(1, 3))
hist(log2(filterStatGlobalMean + 1), breaks = 40, main = "Filter statistic", xlab = "log2(ove
abline(v = log2(5 + 1), col = "red")
plot(density(testStat, na.rm = TRUE), xlab = "Test statistic", main = "Unconditional distribu
plot(density(testStat[keepGlobalMean], na.rm = TRUE), xlab = "Test statistic", main = "Condit
```



- The conditional null distribution is the same as the unconditional one: filtering on the overall mean is valid.
- Not using the group labels is not sufficient: Bourgon et al. (2010) show that filtering on the overall variance invalidates the moderated t-test of limma, because the filter changes the distribution of the variances from which the empirical Bayes prior is estimated. Filtering on the overall mean, as `filterByExpr` does, does not have this problem.

filterByExpr

```
?edgeR::filterByExpr
```

Help page of `filterByExpr`

```
Filter Genes By Expression Level

Description:

     Determine which genes have sufficiently large counts to be
     retained in a statistical analysis.

Usage:

     ## S3 method for class 'DGEList'
     filterByExpr(y, design = NULL, group = NULL, lib.size = NULL, ...)
     ## S3 method for class 'SummarizedExperiment'
     filterByExpr(y, design = NULL, group = NULL, lib.size = NULL, ...)
     ## Default S3 method:
     filterByExpr(y, design = NULL, group = NULL, lib.size = NULL,
                  min.count = 10, min.total.count = 15, large.n = 10, min.prop = 0.7, ...)

Arguments:

       y: matrix of counts, or a 'DGEList' object, or a
          'SummarizedExperiment' object.

  design: design matrix.  Ignored if 'group' is not 'NULL'.  Defaults
          to 'y$design' if 'y' is a DGEList.  Used to compute the
          minimum group sample size ('MinSampleSize').

   group: vector or factor giving group membership.  Provides an
          alternative way to specify the design matrix when there is
          only one explanatory factor (oneway layout).  Defaults to
          'y$samples$group' if 'y' is a DGEList and 'design' and
          'group' are both 'NULL'.

lib.size: library size.  Defaults to 'colSums(y)' if 'y' is a matrix or
          to 'normLibSizes(y)' if 'y' is a DGEList.  Used to compute
          the CPM cutoff value.
```

```
min.count: numeric.  Minimum count required for at least some samples.
          Used to compute the CPM cutoff value.

min.total.count: numeric. Minimum total count required across all
          samples.

 large.n: integer.  Number of samples that would be considered a
          "large" group.  Used to modify 'MinSampleSize' for large
          datasets.

min.prop: numeric.  Used to modify 'MinSampleSize' for large datasets.
          See Details below for the exact formula.

     ...: other arguments are not currently used.

Details:

     This function implements the filtering strategy that was described
     informally by Chen et al (2016). Roughly speaking, the strategy
     keeps genes that have at least 'min.count' reads in a worthwhile
     number samples.

     More precisely, the filtering keeps genes that have 'CPM >=
     CPM.cutoff' in 'MinSampleSize' samples, where 'CPM.cutoff =
     min.count / median(lib.size) * 1e6' and 'MinSampleSize' is the
     smallest group sample size or, more generally, the minimum inverse
     leverage computed from the design matrix.

     If all the group samples sizes are large, then 'MinSampleSize' is
     reduced somewhat. If 'MinSampleSize > large.n', then genes are
     kept if 'CPM >= CPM.cutoff' in 'k' samples where 'k = large.n +
     (MinSampleSize - large.n) * min.prop'. This rule requires that
     genes are expressed in at least 'min.prop * MinSampleSize'
     samples, even when 'MinSampleSize' is large.

     In addition, each kept gene is required to have at least
     'min.total.count' reads across all the samples.

Value:

     Logical vector of length 'nrow(y)' indicating which rows of 'y' to
```

```
     keep in the analysis.

Note:

     Note that 'filterByExpr' implements independent filtering in the
     sense that the filtering would be exactly the same even if the
     sample group labels or the rows of the design matrix were
     permuted.  The function uses the 'design' and 'group' arguments
     only to identify the smallest group size ('MinSampleSize').
     Non-independent filtering that used information about which sample
     belongs to each group, for example keeping genes expressed in a
     certain nmber of samples in at least one group, would be invalid
     because it would potentially cause false discoveries in subsequent
     differential expression analyses.

Author(s):

     Gordon Smyth

References:

     Chen Y, Lun ATL, and Smyth, GK (2016). From reads to genes to
     pathways: differential expression analysis of RNA-Seq experiments
     using Rsubread and the edgeR quasi-likelihood pipeline.
     _F1000Research_ 5, 1438. doi:10.12688/f1000research.8987.2
     <https://doi.org/10.12688/f1000research.8987.2>

Examples:

     ## Not run:

     keep <- filterByExpr(y, design)
     y <- y[keep,]
     ## End(Not run)
```

`filterByExpr` uses the counts per million (CPM), irrespective of the group labels. We reconstruct it for the macrophage data, starting from the unfiltered counts.

```
dgeAll <- DGEListFromTximeta(sg)
keepAuto <- filterByExpr(dgeAll, design)
```

1. A gene needs a CPM corresponding to at least `min.count = 10` reads in a sample with

the median library size.

```
libSize <- dgeAll$samples$lib.size * dgeAll$samples$norm.factors
cpmCutoff <- 10 / median(libSize) * 1e6
cpmCutoff
```

```
[1] 0.257874
```

2. It needs this CPM in at least as many samples as the smallest group. In a design with blocks and a factorial structure there are no simple groups. `filterByExpr` therefore uses the inverse of the largest leverage h_{ii}, the diagonal elements of the hat matrix $\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$. For a one-way design, $1/h_{ii}$ is exactly the size of the group of sample i.

```
leverage <- hat(design)
table(round(1 / leverage, 2))
```

```
2.67
  24
```

```
minSampleSize <- 1 / max(leverage)
```

Every sample has the same leverage here, so a gene needs a sufficient CPM in at least 2.67, i.e. 3, samples.

3. The total count over all samples should be at least `min.total.count = 15`.

```
keepManual <- rowSums(cpm(dgeAll$counts, lib.size = libSize) >= cpmCutoff) >= minSampleSize &
  rowSums(dgeAll$counts) >= 15
table(manual = keepManual, filterByExpr = keepAuto)
```

```
       filterByExpr
manual  FALSE  TRUE
  FALSE 39070     0
  TRUE      0 19224
```

- The manual filter gives exactly the same genes as `filterByExpr`.
- Note that we compute the CPM from the raw counts and the library sizes: `cpm(dgeAll)` would also use the length offsets from tximeta, while `filterByExpr` does not.
- See `?filterByExpr` for the other arguments, e.g. `large.n` and `min.prop` for designs with large groups.

Transcript-level analysis

DGE, DTE and DTU

Salmon quantifies the expression of every transcript. This allows three types of analyses (Soneson et al. 2015):

- **Differential gene expression (DGE)**: does the total expression of the gene change?
- **Differential transcript expression (DTE)**: does the expression of a transcript change?
- **Differential transcript usage (DTU)**: does the relative abundance of the transcripts of a gene change?

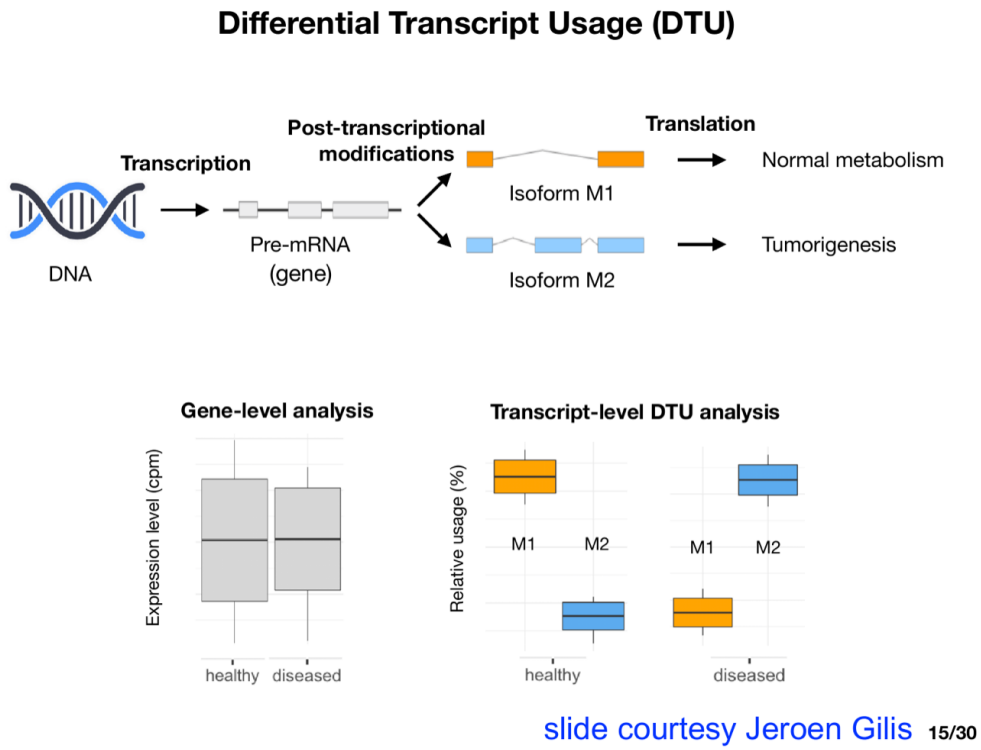


Figure 5: Differential transcript usage: the gene expression stays the same, while the usage of the isoforms switches (slide courtesy of Jeroen Gilis).

Transcript-level data

For the transcript-level analyses we use the naive and IFNγ-primed samples of the macrophage data: a randomised complete block design with the 6 donors as blocks. We restrict the analysis

to one contrast, the effect of IFNγ in uninfected cells, which keeps the DTU and stage-wise analyses simple and limits the computation time: transcript-level data with Gibbs samples are much larger than gene-level data. The salmon quantifications of the macrophage package contain 20 Gibbs samples per sample, which we now need.

```
macData <- coldata |>
  filter(condition %in% c("naive", "IFNg")) |>
  mutate(condition = droplevels(condition))
macData |>
  dplyr::select(names, donor, condition)
```

```
            names  donor condition
1  SAMEA103885102 diku_1     naive
2  SAMEA103885347 diku_1      IFNg
3  SAMEA103885182 eiwy_1     naive
4  SAMEA103885136 eiwy_1      IFNg
5  SAMEA103885368 fikt_3     naive
6  SAMEA103885218 fikt_3      IFNg
7  SAMEA103885284 ieki_2     naive
8  SAMEA103885059 ieki_2      IFNg
9  SAMEA103885111 podx_1     naive
10 SAMEA103884919 podx_1      IFNg
11 SAMEA103885262 qaqx_1     naive
12 SAMEA103885228 qaqx_1      IFNg
```

```
macDesign <- model.matrix(~ donor + condition, data = macData)
```

We import the quantifications with tximeta, which also imports the Gibbs samples as additional assays (`infRep1`, `infRep2`, …).

```
macSt <- tximeta(macData)
assayNames(macSt) |> head()
```

```
[1] "counts"    "abundance" "length"    "infRep1"   "infRep2"   "infRep3"
```

Quantification uncertainty

- Many reads are compatible with several transcripts of the same gene. Their assignment is estimated with the EM algorithm.

- Transcript counts therefore have an additional source of variability: **read-to-transcript ambiguity**. Transcripts that share most of their sequence with other transcripts have a much more uncertain count.
- Quantification uncertainty is therefore especially an issue for transcripts. The assignment of an ambiguous read to one of the transcripts of a gene changes the transcript counts, but not their sum: gene counts are much less affected, and we analyse them without accounting for the quantification uncertainty.
- Salmon and kallisto can quantify this uncertainty with bootstrap or Gibbs samples (inferential replicates), e.g. with `salmon quant ... --numGibbsSamples 20`.
- `catchSalmon` estimates an overdispersion factor for every transcript from the inferential replicates (Baldoni et al. 2024). If a count is c times more variable than expected because of the ambiguity, dividing the count by c gives a **scaled count** with the usual mean-variance relation, which can be analysed with the standard edgeR pipeline.

Why does dividing work? Suppose that the ambiguity inflates the technical variance of the count Y_{ti} of transcript t by a factor c_t:

$$\mathrm{Var}(Y_{ti}) \approx c_t \, \mu_{ti}.$$

The scaled count $\tilde{Y}_{ti} = Y_{ti}/c_t$ then has

$$E[\tilde{Y}_{ti}] = \frac{\mu_{ti}}{c_t}, \quad \mathrm{Var}(\tilde{Y}_{ti}) = \frac{c_t \, \mu_{ti}}{c_t^2} = \frac{\mu_{ti}}{c_t} = E[\tilde{Y}_{ti}],$$

so it again has a Poisson-like technical variability, and biological variability can be modelled on top of it as usual.

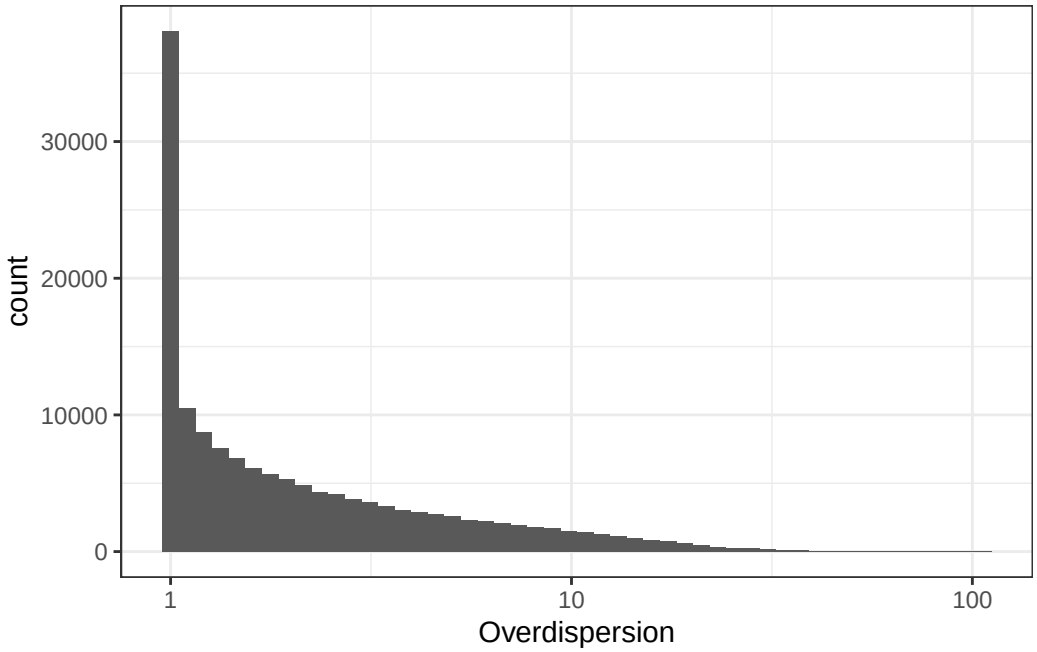
- The scaled count is an **effective count**: a transcript whose count is 4 times too variable carries the same information as a count that is 4 times smaller.
- The factor c_t is estimated from the inferential replicates: within every sample the variance of the bootstrap or Gibbs counts is compared with their mean, this ratio is pooled over the samples and moderated, and it is at least 1.
- The idea is similar to that of the juggling offsets below: both make the standard bulk RNA-seq tools applicable by changing their input. Scaling changes the counts, juggling offsets change the offset.

`catchSalmon` reads the salmon output directly. When the data are imported with tximeta, `DGEListFromTximeta` does the same: it estimates the overdispersion factors from the inferential replicates and, with `divide = TRUE`, returns the divided counts (Baldoni et al. 2024, 2025).

```
macRaw <- DGEListFromTximeta(macSt)
macDivided <- DGEListFromTximeta(macSt, divide = TRUE)
summary(macDivided$genes$Overdispersion)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.000   1.000   1.104   2.620   2.335 106.408
```

```
tibble(Overdispersion = macDivided$genes$Overdispersion) |>
  filter(Overdispersion > 1) |>
  ggplot(aes(Overdispersion)) +
  geom_histogram(bins = 50) +
  scale_x_log10() +
  theme_bw()
```



Some transcripts have counts that are many times more variable than their expression level suggests.

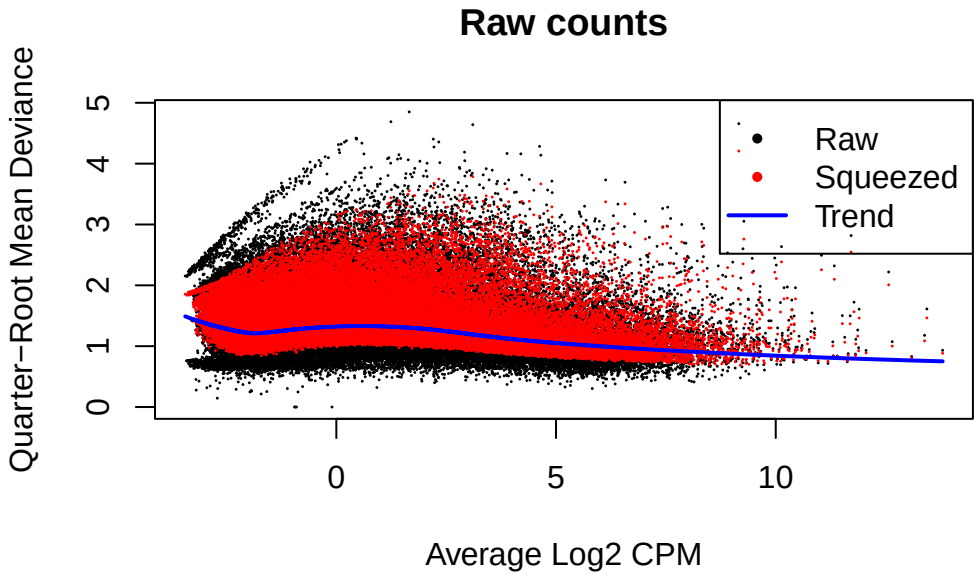
Long reads (PacBio, Oxford Nanopore) sequence full-length transcripts and thus remove most of the read-to-transcript ambiguity. Their lower throughput and higher error rate, however, imply that short reads remain the standard for quantification.

Differential transcript expression

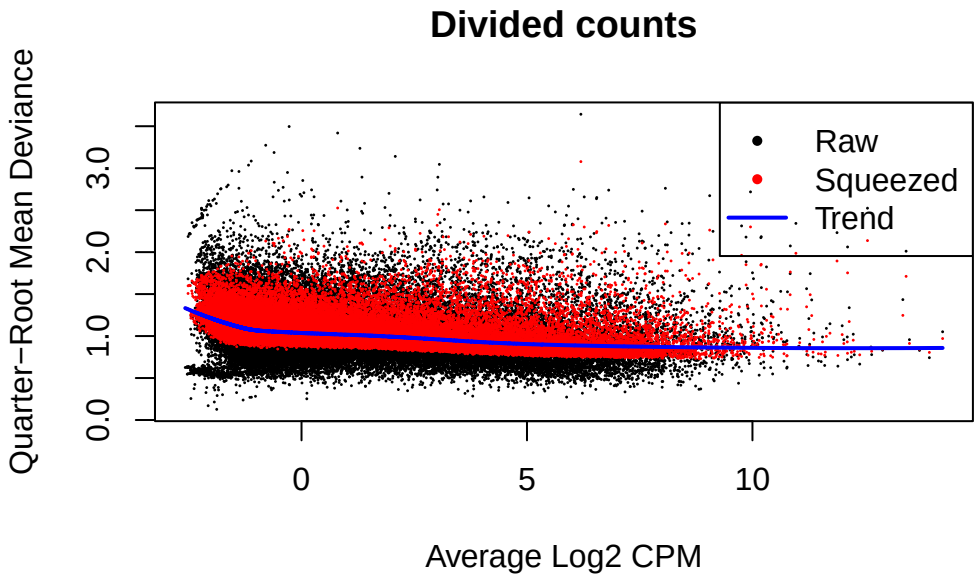
We analyse the raw and the divided transcript counts with the same edgeR pipeline.

```
dteAnalysis <- function(y) {
  y$genes <- data.frame(tx = y$genes$tx_name, gene = y$genes$gene_id,
                        Overdispersion = y$genes$Overdispersion, row.names = rownames(y))
  keep <- filterByExpr(y, macDesign)
  y <- y[keep, , keep.lib.sizes = FALSE]
  y <- normLibSizes(y)
  fit <- glmQLFit(y, macDesign)
  list(y = y, fit = fit, test = glmQLFTest(fit, coef = "conditionIFNg"))
}
dteRaw <- dteAnalysis(macRaw)
dteScaled <- dteAnalysis(macDivided)
```

```
plotQLDisp(dteRaw$fit, main = "Raw counts")
```




```
plotQLDisp(dteScaled$fit, main = "Divided counts")
```




```
c(raw = median(dteRaw$fit$s2.post), divided = median(dteScaled$fit$s2.post))
```

```
     raw  divided
2.934842 1.152101
```

- With the raw counts, the QL dispersions are inflated: the ambiguity overdispersion is not captured by the NB model.
- After dividing by the overdispersion, the QL dispersions are centred around one.

```
summary(decideTests(dteScaled$test))
```

```
       conditionIFNg
Down            7460
NotSig         36607
Up              7833
```

Differential transcript usage

The usage of transcript t of gene g in sample i is its fraction of the gene's total count, i.e. the fraction of the fragments of the gene that come from the transcript:

$$\pi_{gti} = \frac{E[Y_{gti}]}{E[Y_{g.i}]}, \quad Y_{g.i} = \sum_t Y_{gti}.$$

Only genes with at least two transcripts can show DTU.

```
yTx <- dteScaled$y
multiTxGenes <- yTx$genes |>
  dplyr::count(gene) |>
  filter(n > 1) |>
  pull(gene)
dtu <- yTx[yTx$genes$gene %in% multiTxGenes, ]
c(genes = length(multiTxGenes), transcripts = nrow(dtu))
```

```
genes transcripts
10731       44973
```

Juggling offsets

In DGE we use the (effective) library size as offset, so that the parameters model the expression relative to the sequencing depth. For DTU we **replace this offset by the total count of the gene** (Segers et al. 2023):

$$\begin{cases} E[Y_{gti} \mid Y_{g.i}] &= \mu_{gti} \\ \log \mu_{gti} &= \mathbf{x}_i^T \beta_{gt} + \log Y_{g.i} \\ \mathrm{Var}[Y_{gti} \mid Y_{g.i}] &= \sigma^2_{gt}(\mu_{gti} + \phi \mu^2_{gti}) \end{cases} \quad \Rightarrow \quad \log \pi_{gti} = \log \frac{\mu_{gti}}{Y_{g.i}} = \mathbf{x}_i^T \beta_{gt}.$$

- We model the counts conditionally on the observed gene total.
- The parameters now model the log usage of the transcript.
- A condition effect in β_{gt} is a change in usage: DTU.
- The gene total already accounts for the sequencing depth: no further normalisation is needed.
- Everything else in the edgeR pipeline stays the same: QL dispersions, empirical Bayes and moderated F-tests. The same trick works with DESeq2.
- The gene total is plugged in as known; its uncertainty is ignored.
- The total count offset above would work for raw transcript counts.

Here, we model the divided counts $\tilde{Y}_{gti} = Y_{gti}/c_t$, so that the quantification uncertainty of the transcripts is accounted for.

- We model the counts conditionally on the observed gene total $Y_{g.i}$.
- Given the gene total, their mean is the usage times the gene total, divided by c_t:

$$E[\tilde{Y}_{gti} \mid Y_{g.i}] = \frac{\pi_{gti}\, Y_{g.i}}{c_t} \quad \Rightarrow \quad \text{offset} = \log Y_{g.i} - \log c_t,$$

- with π_{gti} the usage of transcript t, i.e. the fraction of the fragments of gene g that come from transcript t.
- The gene total $Y_{g.i}$ is computed from the **undivided** counts.
- The undivided gene total is hardly affected by the quantification uncertainty: the ambiguity between transcripts of the same gene cancels out in the sum.
- The term $-\log c_t$ is the same in all samples. It only changes the intercept of the transcript, not the log fold changes.

The undivided counts are the divided counts multiplied by the overdispersion.

```
undivided <- dtu$counts * dtu$genes$Overdispersion
geneTotals <- rowsum(undivided, dtu$genes$gene)[dtu$genes$gene, ]
dtu$offset <- log(pmax(geneTotals, 1)) - log(dtu$genes$Overdispersion)

fitDtu <- glmQLFit(dtu, macDesign)
qlfDtu <- glmQLFTest(fitDtu, coef = "conditionIFNg")
topTags(qlfDtu, n = 5)
```

```
Coefficient:  conditionIFNg
                                  tx                gene Overdispersion
ENST00000381318.7 ENST00000381318.7 ENSG00000205726.14       1.488381
ENST00000572992.1 ENST00000572992.1  ENSG00000175643.9       1.110110
ENST00000482504.1 ENST00000482504.1 ENSG00000097033.14       1.434778
ENST00000249700.8 ENST00000249700.8  ENSG00000128872.9       1.274763
ENST00000358691.9 ENST00000358691.9 ENSG00000198265.11       1.782383
                      logFC   logCPM        F       PValue          FDR
ENST00000381318.7 -2.2703021 4.981862 988.8985 4.218109e-11 1.054087e-06
ENST00000572992.1  5.3582922 3.266962 435.3478 4.687643e-11 1.054087e-06
ENST00000482504.1  0.4430633 5.951837 870.6890 7.755396e-11 1.162611e-06
ENST00000249700.8 -0.3420956 6.154322 594.9536 4.822964e-10 5.422579e-06
ENST00000358691.9 -0.3679648 6.040809 497.7016 1.132123e-09 1.018299e-05
```

```
summary(decideTests(qlfDtu))
```

```
       conditionIFNg
Down            1317
NotSig         42558
Up              1098
```

The estimated global NB dispersion is close to zero:

```
fitDtu$dispersion
```

```
[1] 0.0002862964
```

- Given the gene total, the transcript counts are no longer overdispersed by the biological variation of the gene expression, which cancels out in the usage.
- The model then reduces to a quasi-Poisson model on the usages, close to the quasi-binomial model of the dedicated DTU tool satuRn.

Fragment usage or molecular usage?

- The counts are numbers of fragments.
- Let M_{gti} be the (unobserved) number of molecules of transcript t of gene g in the library of sample i, and
- $M_{g.i} = \sum_{t'} M_{gt'i}$ the numbers of molecules of all transcripts of the gene.
- The **molecular usage** is the fraction of the molecules of the gene that are transcript t,

$$\rho_{gti} = M_{gti}/M_{g.i}$$

A transcript produces fragments in proportion to its number of molecules times its effective length l_{ti}:

$$E[Y_{gti} \mid M_{g.i}] = k_i \, l_{ti} \, M_{gti} \; = k_i \, l_{ti} \, \rho_{gti} \, M_{g.i},$$

- with k_i the expected number of sequenced fragments per molecule and per base in sample i.
- It depends on the sequencing depth and on the composition of the library, but it is the same for all transcripts in the sample.

- Dividing by the effective length and summing over the transcripts of the gene gives

$$\sum_{t'} E[Y_{gt'i} \mid M_{g\cdot i}]/l_{t'i} = k_i M_{g.i},$$

  which we estimate by the length-normalised gene total $\sum_{t'} Y_{gt'i}/l_{t'i}$.

Hence

$$\log \mu_{gti} = \log \rho_{gti} + \log l_{ti} + \log \sum_{t'} \frac{Y_{gt'i}}{l_{t'i}} :$$

- with $\log l_{ti} + \log \sum_{t'} Y_{gt'i}/l_{t'i}$ as offset, i.e. the log of the effective length of the transcript multiplied by the length-normalised gene total
- The remaining model parameters of the mean model thus describe the log molecular usage,

$$\log \rho_{gti} = \mathbf{x}_i^T \beta_{gt}$$

With the gene total $Y_{g.i}$ as offset, we instead model the **fragment usage**: the expected fraction of the fragments of the gene that come from transcript t.

$$\pi_{gti} = \frac{l_{ti}\, \rho_{gti}}{\sum_{t'} l_{t'i}\, \rho_{gt'i}}.$$

Indeed,

$$E[Y_{g.i} \mid M_{g\cdot i}] = k_i \sum_{t'} l_{t'i}\, M_{gt'i} \; = k_i\, M_{g.i} \sum_{t'} l_{t'i}\, \rho_{gt'i}$$

- If a gene shifts towards longer transcripts, the fragment usage of a transcript whose molecular usage does not change, also decreases. Tests on the fragment usage can thus flag transcripts that do not change themselves.
- The molecular usage is the quantity of biological interest.
- Because salmon's TPM is proportional to Y_{gti}/l_{ti}, the estimated molecular usage $\hat{\rho}_{gti} = (Y_{gti}/l_{ti})/\sum_{t'} Y_{gt'i}/l_{t'i}$ is the TPM of the transcript divided by the TPM of the gene; as for the gene total, the sum uses the undivided counts, and $-\log c_t$ is added for the divided counts.
- This is the offset counterpart of `countsFromAbundance = "dtuScaledTPM"` in tximport (Love et al. 2018).

- The gene-level null hypothesis of no DTU is the same on both scales, but the transcript-level tests and log fold changes differ for genes whose transcripts differ strongly in length.

We fit the offset model for the molecular usage on the macrophage data:

```
effLength <- assay(macSt, "length")[rownames(dtu), ]
lengthNormTotals <- rowsum(undivided / effLength, dtu$genes$gene)[dtu$genes$gene, ]

dtuLength <- dtu
# as for the gene totals, the offset of a gene with zero counts in a sample is set to 0
dtuLength$offset <- log(effLength) + log(pmax(lengthNormTotals, 1 / effLength)) -
  log(dtu$genes$Overdispersion)

fitDtuLength <- glmQLFit(dtuLength, macDesign)
qlfDtuLength <- glmQLFTest(fitDtuLength, coef = "conditionIFNg")
table(fragment = p.adjust(qlfDtu$table$PValue, "BH") < 0.05,
      molecular = p.adjust(qlfDtuLength$table$PValue, "BH") < 0.05)
```

```
        molecular
fragment FALSE  TRUE
   FALSE 42295   263
   TRUE    598  1817
```

- Most transcripts that are significant on one scale are also significant on the other, but a substantial fraction is only significant on one of both scales: these are transcripts of genes whose average transcript length changes between the conditions.
- The transcript-level tutorial compares both offsets in more detail on the airway data, with a check that the difference between both log fold changes equals the change in the average transcript length of the gene.

diffSplice

edgeR and limma also have a dedicated function for differential usage. `diffSplice` uses the transcript-level fit with the usual offsets and tests whether the log fold change of a transcript differs from the log fold change of the gene as a whole. This is a test for a transcript × condition interaction within each gene. In recent versions of edgeR and limma, `diffSplice` is a generic function that works on both edgeR (`glmQLFit`) and limma (`voomLmFit`) fits; for an edgeR fit it calls the method `diffSplice.DGEGLM` (see `?diffSplice.DGEGLM`). It replaces the older `diffSpliceDGE` (Baldoni et al. 2025).

```
fitTx <- dteScaled$fit
ds <- diffSplice(fitTx[fitTx$genes$gene %in% multiTxGenes, ],
                 coef = "conditionIFNg", geneid = "gene", exonid = "tx", verbose = FALSE)
topSplice(ds, test = "t", number = 5)
```

```
                                tx               gene Overdispersion     logFC
ENST00000557239.5 ENST00000557239.5 ENSG00000140105.17      10.363184 -
3.749029
ENST00000554772.5 ENST00000554772.5 ENSG00000140105.17       3.568620  2.876456
ENST00000544944.6 ENST00000544944.6 ENSG00000104859.14       3.029913  2.033541
ENST00000435136.6 ENST00000435136.6 ENSG00000087266.15      15.946682  6.513229
ENST00000490682.5 ENST00000490682.5 ENSG00000064012.21       1.256303  2.196197
                          t      P.Value          FDR
ENST00000557239.5 -12.98560 4.096922e-27 1.842509e-22
ENST00000554772.5  11.54364 4.791145e-23 1.077361e-18
ENST00000544944.6  14.49752 8.389585e-20 1.233792e-15
ENST00000435136.6  11.73338 1.097363e-19 1.233792e-15
ENST00000490682.5  12.62494 2.872280e-18 2.583501e-14
```

```
topSplice(ds, test = "F", number = 5)
```

```
                                gene NExons        F      P.Value          FDR
ENSG00000140105.17 ENSG00000140105.17     28 20.26873 1.105625e-39 1.186446e-
35
ENSG00000065882.15 ENSG00000065882.15      8 48.66688 3.025609e-21 1.623390e-
17
ENSG00000087266.15 ENSG00000087266.15     17 16.80134 3.019978e-20 1.080246e-
16
ENSG00000104859.14 ENSG00000104859.14      9 26.85855 8.261721e-16 2.216413e-
12
ENSG00000064012.21 ENSG00000064012.21      9 23.27945 1.838255e-15 3.945264e-
12
```

```
topSplice(ds, test = "simes", number = 5)
```

```
                                gene NExons      P.Value          FDR
ENSG00000140105.17 ENSG00000140105.17     28 1.147138e-25 1.230994e-21
ENSG00000104859.14 ENSG00000104859.14      9 7.550626e-19 4.051288e-15
ENSG00000087266.15 ENSG00000087266.15     17 1.865516e-18 6.672952e-15
ENSG00000064012.21 ENSG00000064012.21      9 2.585052e-17 6.935048e-14
ENSG00000065882.15 ENSG00000065882.15      8 3.808713e-17 8.174261e-14
```

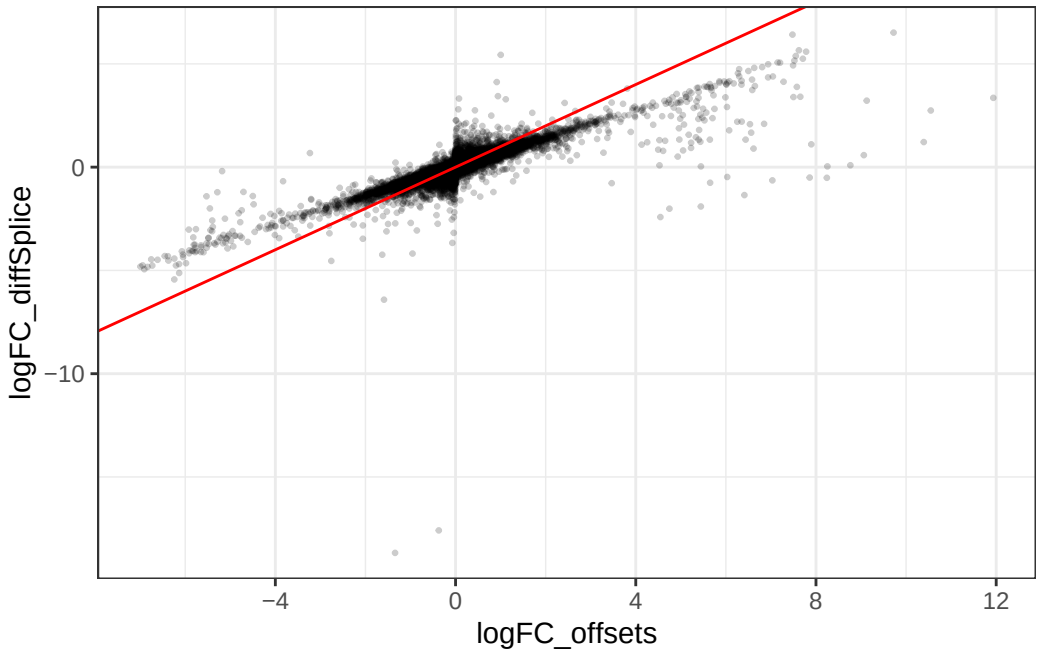
- `test = "t"`: transcript-level tests.
- `test = "F"`: gene-level F-test of any DTU within the gene. It is most powerful when several transcripts change in opposite directions.
- `test = "simes"`: gene-level test that combines the transcript-level p-values with the Simes method. It is most powerful when one transcript stands out.

In the simulation study of Baldoni et al. (2025), the edgeR and limma pipelines on divided counts controlled the FDR in all scenarios, also for small sample sizes, and edgeR with Simes p-values was the most powerful.

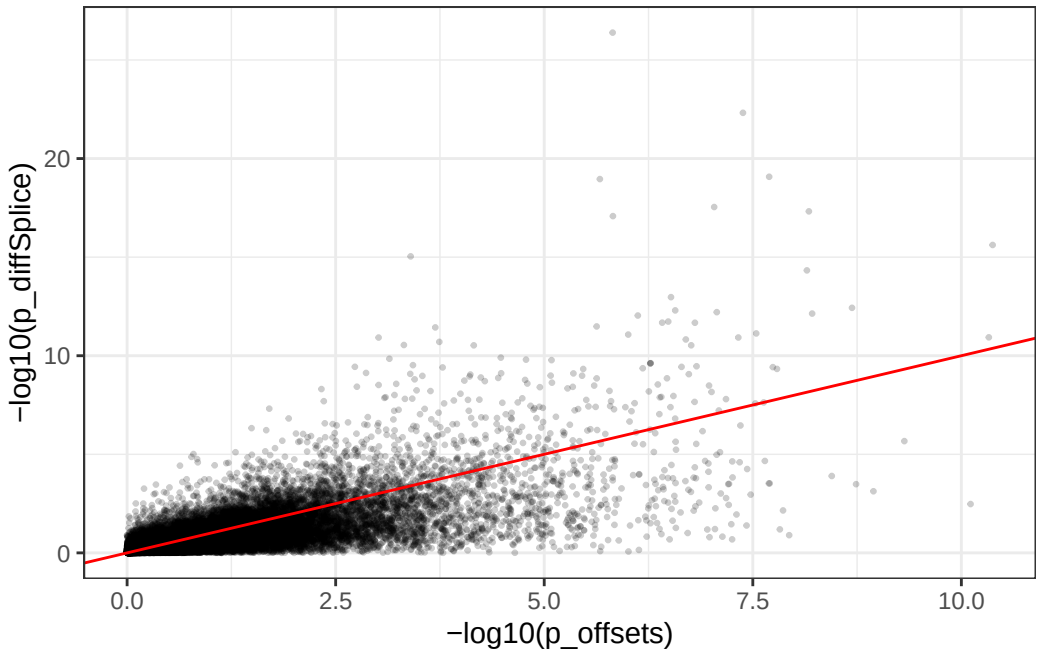
We compare both approaches on the transcripts that are analysed by both.

```
dtuComparison <- inner_join(
  tibble(tx = dtu$genes$tx, logFC_offsets = qlfDtu$table$logFC, p_offsets = qlfDtu$table$PVal
  tibble(tx = ds$genes$tx, logFC_diffSplice = as.vector(ds$coefficients), p_diffSplice = as.v
  by = "tx"
) |>
  mutate(
    offsets = p.adjust(p_offsets, "BH") < 0.05,
    diffSplice = p.adjust(p_diffSplice, "BH") < 0.05
  )

dtuComparison |>
  ggplot(aes(logFC_offsets, logFC_diffSplice)) +
  geom_point(alpha = 0.2, size = 0.5) +
  geom_abline(intercept = 0, slope = 1, col = "red") +
  theme_bw()
```




```
dtuComparison |>
  ggplot(aes(-log10(p_offsets), -log10(p_diffSplice))) +
  geom_point(alpha = 0.2, size = 0.5) +
  geom_abline(intercept = 0, slope = 1, col = "red") +
  theme_bw()
```




```
c(
  logFC = cor(dtuComparison$logFC_offsets, dtuComparison$logFC_diffSplice, method = "spearman
  pValue = cor(dtuComparison$p_offsets, dtuComparison$p_diffSplice, method = "spearman")
)
```

```
    logFC    pValue
0.9094455 0.7773737
```

```
table(offsets = dtuComparison$offsets, diffSplice = dtuComparison$diffSplice)
```

```
       diffSplice
offsets FALSE  TRUE
  FALSE 42089   469
  TRUE   1510   905
```

Both approaches agree well on the direction and size of the usage changes, but only moderately on the ranking. The offset model estimates a dispersion per transcript, while diffSplice shares one dispersion across all transcripts of a gene, so transcripts with a variability that differs from the rest of their gene are ranked differently.

Dedicated DTU methods also exist, e.g. DEXSeq, DRIMSeq and satuRn.

Stage-wise testing

DTU analyses test many transcripts per gene. Biologists are mostly interested in

1. which genes show DTU, and
2. which transcripts within these genes are responsible.

stageR (Van den Berge et al. 2017) controls the FDR at the gene level with a two-stage procedure:

1. **Screening stage**: test every gene for DTU (any transcript) and control the FDR over genes.
2. **Confirmation stage**: for the significant genes only, test the individual transcripts with a family-wise error rate correction within the gene.

We perform two stage-wise analyses, each of which uses a single DTU method for both stages:

1. **diffSplice**: the gene-level Simes p-values of `diffSplice` for screening and its transcript-level tests for confirmation.
2. **Offset model**: the Simes method applied to the transcript-level p-values of the offset model for screening, and the same transcript-level p-values for confirmation.

The Simes p-value of a gene with n transcripts combines the ordered transcript-level p-values $p_{(1)} \leq ... \leq p_{(n)}$ into $\min_k n\, p_{(k)}/k$.

```
simes <- function(p) min(sort(p) * length(p) / seq_along(p))

stageWiseDtu <- function(pScreen, pTranscript, tx, gene) {
  pConfirmation <- matrix(pTranscript, ncol = 1, dimnames = list(tx, "transcript"))
  stageRObj <- stageRTx(pScreen = pScreen, pConfirmation = pConfirmation,
                        tx2gene = data.frame(transcript = tx, gene = gene))
  stageRObj <- stageWiseAdjustment(stageRObj, method = "dtu", alpha = 0.05)
  getAdjustedPValues(stageRObj, order = TRUE, onlySignificantGenes = TRUE)
}

padjDs <- stageWiseDtu(
  pScreen = setNames(as.vector(ds$gene.simes.p.value), ds$gene.genes$gene),
  pTranscript = as.vector(ds$p.value), tx = ds$genes$tx, gene = ds$genes$gene
)
padjOffset <- stageWiseDtu(
  pScreen = c(tapply(qlfDtu$table$PValue, dtu$genes$gene, simes)),
  pTranscript = qlfDtu$table$PValue, tx = dtu$genes$tx, gene = dtu$genes$gene
)
padjLength <- stageWiseDtu(
```

```
  pScreen = c(tapply(qlfDtuLength$table$PValue, dtu$genes$gene, simes)),
  pTranscript = qlfDtuLength$table$PValue, tx = dtu$genes$tx, gene = dtu$genes$gene
)
head(padjDs)
```

```
              geneID              txID         gene   transcript
1 ENSG00000140105.17 ENST00000557239.5 1.230994e-21 1.427049e-24
2 ENSG00000140105.17 ENST00000554772.5 1.230994e-21 1.668862e-20
3 ENSG00000140105.17 ENST00000344102.9 1.230994e-21 2.893601e-15
4 ENSG00000140105.17 ENST00000554331.5 1.230994e-21 7.050184e-10
5 ENSG00000140105.17 ENST00000355338.6 1.230994e-21 8.107707e-03
6 ENSG00000140105.17 ENST00000556504.5 1.230994e-21 2.132143e-02
```

```
head(padjOffset)
```

```
              geneID              txID         gene   transcript
1 ENSG00000097033.14 ENST00000482504.1 1.664463e-06 1.364314e-09
2 ENSG00000097033.14 ENST00000370558.8 1.664463e-06 2.414491e-07
3 ENSG00000097033.14 ENST00000535010.5 1.664463e-06 1.000000e+00
4 ENSG00000097033.14 ENST00000602603.1 1.664463e-06 1.000000e+00
5  ENSG00000175643.9 ENST00000572992.1 1.664463e-06 8.246409e-10
6  ENSG00000175643.9 ENST00000312499.5 1.664463e-06 4.731961e-06
```

```
rbind(
  diffSplice = c(genes = length(unique(padjDs$geneID)), transcripts = sum(padjDs$transcript <
  offset = c(genes = length(unique(padjOffset$geneID)), transcripts = sum(padjOffset$transcr
  both = c(genes = length(intersect(padjDs$geneID, padjOffset$geneID)),
           transcripts = length(intersect(padjDs$txID[padjDs$transcript < 0.05],
                                          padjOffset$txID[padjOffset$transcript < 0.05]))),
  `offset, molecular usage` = c(genes = length(unique(padjLength$geneID)),
                                transcripts = sum(padjLength$transcript < 0.05)),
  `offset, fragment and molecular` = c(genes = length(intersect(padjOffset$geneID, padjLengt
           transcripts = length(intersect(padjOffset$txID[padjOffset$transcript < 0.05],
                                          padjLength$txID[padjLength$transcript < 0.05])))
)
```

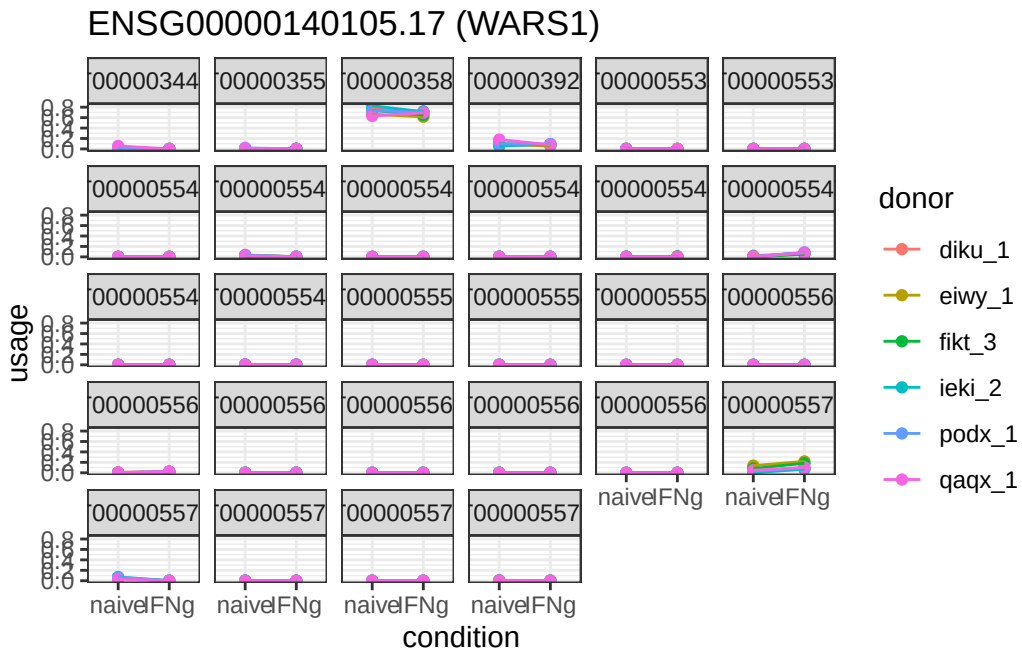
```
                              genes transcripts
diffSplice                      801        1193
offset                         1220        2020
both                            582         780
```

```
offset, molecular usage         1238        1855
offset, fragment and molecular  1077        1570
```

- Both stage-wise analyses find many genes with DTU, but they agree only moderately: they test a different quantity and estimate the variability differently.
- `diffSplice` compares the log fold change of each transcript with that of the gene as a whole in the transcript-level DTE model. The offset model plugs in the observed gene total as a known offset and models the usage directly, with a separate QL dispersion for the usage of every transcript.
- The offset models for the fragment and the molecular usage select almost the same number of genes, with a large overlap, but they differ more at the transcript level: the gene-level null hypothesis is the same on both scales, the transcript-level tests are not.
- On these data the offset model finds more genes, while `diffSplice` was the most powerful method in the simulation study of Baldoni et al. (2025). In the transcript-level tutorial on the airway data, `diffSplice` finds more genes: which method is the most powerful depends on the data.

We plot the usages of the transcripts of the top gene of the `diffSplice` analysis, *WARS1*, a gene that is strongly induced by interferon gamma:

```
topGene <- padjDs$geneID[1]
usage <- undivided[dtu$genes$gene == topGene, ] /
  geneTotals[dtu$genes$gene == topGene, ]
usage |>
  as_tibble(rownames = "tx") |>
  pivot_longer(-tx, names_to = "names", values_to = "usage") |>
  left_join(macData, by = "names") |>
  ggplot(aes(condition, usage, group = donor, color = donor)) +
  geom_point() +
  geom_line() +
  facet_wrap(~ tx) +
  ggtitle(paste0(topGene, " (", mapIds(org.Hs.eg.db, sub("\\..*", "", topGene), "SYMBOL", "E
  theme_bw()
```



Summary

- GLMs are fitted with IRLS: iterated weighted least squares fits of a working response. Inference with Wald and likelihood ratio tests is asymptotic.
- Quasi-likelihood only specifies the mean and the variance. The QL dispersion is estimated from the deviance and leads to t- and F-tests.
- edgeR 4 combines a global NB dispersion with gene-specific QL dispersions with bias-adjusted degrees of freedom, moderated with empirical Bayes: the same strategy as limma and msqrob2.
- Normalisation methods (TMM, median of ratios) assume that most genes are not DE and provide offsets.
- Filtering is only valid if the filter statistic is independent of the test statistic under H_0 (independent filtering).
- At the transcript level, read-to-transcript ambiguity adds variability, which we divide out with the overdispersion estimated from the Gibbs samples. DTU can be analysed with the standard tools by replacing the library size offset with the gene totals; adding a length correction to the offset models the molecular instead of the fragment usage. Stage-wise testing controls the FDR at the gene level.

References

Alasoo, Kaur, Julia Rodrigues, Subhankar Mukhopadhyay, et al. 2018. "Shared Genetic Effects on Chromatin and Gene Expression Indicate a Role for Enhancer Priming in Immune Response." *Nature Genetics* 50 (3): 424–31.

Baldoni, Pedro L, Lizhong Chen, Mengbo Li, Yunshun Chen, and Gordon K Smyth. 2025. "Dividing Out Quantification Uncertainty Enables Assessment of Differential Transcript Usage with Limma and edgeR." *Nucleic Acids Research* 53 (22): gkaf1305. https://doi.org/10.1093/nar/gkaf1305.

Baldoni, Pedro L, Yunshun Chen, Soroor Hediyeh-zadeh, et al. 2024. "Dividing Out Quantification Uncertainty Allows Efficient Assessment of Differential Transcript Expression with edgeR." *Nucleic Acids Research* 52 (3): e13.

Bourgon, Richard, Robert Gentleman, and Wolfgang Huber. 2010. "Independent Filtering Increases Detection Power for High-Throughput Experiments." *Proceedings of the National Academy of Sciences* 107 (21): 9546–51.

Chen, Yunshun, Lizhong Chen, Aaron T L Lun, Pedro L Baldoni, and Gordon K Smyth. 2025. "edgeR V4: Powerful Differential Analysis of Sequencing Data with Expanded Functionality and Improved Support for Small Counts and Larger Datasets." *Nucleic Acids Research* 53 (2): gkaf018.

Law, Charity W, Yunshun Chen, Wei Shi, and Gordon K Smyth. 2014. "Voom: Precision Weights Unlock Linear Model Analysis Tools for RNA-seq Read Counts." *Genome Biology* 15 (2): R29.

Love, Michael I, Wolfgang Huber, and Simon Anders. 2014. "Moderated Estimation of Fold Change and Dispersion for RNA-seq Data with DESeq2." *Genome Biology* 15 (12): 550.

Love, Michael I., Charlotte Soneson, and Rob Patro. 2018. "Swimming Downstream: Statistical Analysis of Differential Transcript Usage Following Salmon Quantification." *F1000Research* 7: 952. https://doi.org/10.12688/f1000research.15398.3.

Lund, Steven P, Dan Nettleton, Davis J McCarthy, and Gordon K Smyth. 2012. "Detecting Differential Expression in RNA-sequence Data Using Quasi-Likelihood with Shrunken Dispersion Estimates." *Statistical Applications in Genetics and Molecular Biology* 11 (5).

McCarthy, Davis J, Yunshun Chen, and Gordon K Smyth. 2012. "Differential Expression Analysis of Multifactor RNA-Seq Experiments with Respect to Biological Variation." *Nucleic Acids Research* 40 (10): 4288–97.

Robinson, Mark D, and Alicia Oshlack. 2010. "A Scaling Normalization Method for Differential Expression Analysis of RNA-seq Data." *Genome Biology* 11 (3): R25.

Segers, Alexandre, Jeroen Gilis, Mattias Van Heetvelde, Davide Risso, Elfride De Baere, and Lieven Clement. 2023. "Juggling Offsets Unlocks RNA-seq Tools for Fast and Scalable Differential Usage, Aberrant Splicing and Expression Retrieval." *bioRxiv*, 2023.06.29.547014.

Soneson, Charlotte, Michael I Love, and Mark D Robinson. 2015. "Differential Analyses for RNA-seq: Transcript-Level Estimates Improve Gene-Level Inferences." *F1000Research* 4: 1521.

Van den Berge, Koen, Charlotte Soneson, Mark D Robinson, and Lieven Clement. 2017. "stageR: A General Stage-Wise Method for Controlling the Gene-Level False Discovery Rate in Differential Expression and Differential Transcript Usage." *Genome Biology* 18: 151.